

UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL MENDOZA

Tecnicatura Universitaria en Programación

**Automatización inteligente del registro de incidentes
en mesas de ayuda empresariales mediante
orquestración de
flujos con N8N y procesamiento de lenguaje natural
basado en modelos de lenguaje grandes**

Trabajo Final de Carrera presentado para obtener el título de

Técnico Universitario en Programación

Autores

Luca Gomez · Carla Bustos · Gonzalo Sevilla

Director

Prof. Alberto Cortez

Mendoza, República Argentina

2026

Resumen

La creciente digitalización de los procesos organizacionales ha generado una dependencia cada vez mayor de los sistemas informáticos para la ejecución de las actividades operativas. En este contexto, la gestión eficiente de incidentes tecnológicos constituye un factor determinante para garantizar la continuidad del negocio y la calidad del servicio brindado a los usuarios internos. Sin embargo, en numerosas organizaciones medianas el proceso de registro inicial de incidentes continúa realizándose de manera manual, lo que introduce demoras operativas, errores de clasificación y una utilización ineficiente de los recursos técnicos disponibles.

La presente investigación tiene como propósito el diseño, desarrollo, implementación y validación experimental de un sistema automatizado de mesa de ayuda empresarial basado en la integración de orquestación de flujos mediante la plataforma N8N, un módulo de procesamiento de lenguaje natural desarrollado en Python con el marco FastAPI, una capa de inferencia lingüística sustentada en el modelo Gemini 2.5 de Google, una capa de persistencia sobre PostgreSQL versión 15 y un canal de telefonía gestionado por Twilio. La arquitectura propuesta contempla la recepción de solicitudes a través de tres canales paralelos —correo electrónico, formulario web y llamadas telefónicas con transcripción automática— y deriva cada incidente al sector responsable (Sistemas, Operaciones o Soporte Técnico) mediante un clasificador híbrido en dos etapas que combina reglas determinísticas y modelo de lenguaje grande.

El estudio se desarrolló bajo un enfoque cuantitativo con diseño cuasiexperimental de muestras pareadas y contrabalanceo del orden de procesamiento. El corpus de validación, construido por muestreo estratificado proporcional sobre un universo trimestral de aproximadamente 3.700 incidentes, comprende 200 casos representativos del entorno operativo. El acuerdo entre los dos analistas que realizaron el doble etiquetado independiente alcanzó un coeficiente Kappa de Cohen de 0,87, considerado sustancial.

Los resultados obtenidos evidencian una reducción estadísticamente significativa del tiempo medio de registro, que descendió desde 165,3 s en el flujo manual hasta 18,2 s en el flujo automatizado, lo que representa una reducción relativa del 89 %. La prueba de Wilcoxon de rangos con signo aplicada sobre las mediciones pareadas arrojó un valor $p < 0,001$, permitiendo rechazar la hipótesis nula de igualdad de medianas. La exactitud global del clasi-

ficador alcanzo el 92 % y el valor F1 macro promediado se ubico en 0,919. La proporcion de intervencion humana descendio desde el 100 % en el flujo manual hasta el 9,5 % en el flujo automatizado, materializando un esquema *human in the loop* en el cual el operador conserva la potestad de revision sobre los casos de baja confianza o ambigüedad estructural.

Los hallazgos confirman la viabilidad tecnica, economica y operativa de la automatizacion inteligente del registro de tickets en organizaciones medianas. La investigacion aporta una arquitectura reproducible y escalable construida sobre componentes maduros de código abierto e inferencia linguistica externa, evidencia empirica sobre el desempeño de modelos de lenguaje grandes aplicados a clasificacion de tickets en español rioplatense, y un marco normativo aplicable que contempla la Ley 25.326 de Proteccion de los Datos Personales de la Republica Argentina.

Palabras clave: automatizacion, mesa de ayuda, procesamiento de lenguaje natural, modelo de lenguaje grande, N8N, FastAPI, clasificacion de tickets, español rioplatense, human in the loop

Abstract

The increasing digitization of organizational processes has made IT incident management critical for business continuity. Yet in many medium-sized organizations, incident registration remains entirely manual, introducing delays, classification errors, and inefficient use of technical resources. This thesis presents the design, implementation, and experimental validation of an automated help desk system integrating N8N workflow orchestration, a Python-based natural language processing module built with FastAPI, Google's Gemini 2.5 Flash large language model, PostgreSQL for persistence, and Twilio for telephony. The architecture receives requests through three parallel channels —email, web form, and phone calls with automatic transcription— and routes each incident to the appropriate sector using a two-stage hybrid classifier that combines deterministic rules with a large language model.

The study employed a quantitative quasi-experimental paired-samples design with a stratified proportional sample of 200 incidents drawn from a quarterly universe of approximately 3,700. Inter-rater agreement reached a Cohen's Kappa of 0.87. Results show a statistically significant reduction in mean registration time from 165.3 s to 18.2 s —an 89 % reduction— with a Wilcoxon signed-rank test yielding $p < 0.001$ and maximum effect size ($r = 1.00$). Classification accuracy reached 92 % with a macro-averaged F1 score of 0.919. Human intervention decreased from 100 % to 9.5 %, establishing a *human-in-the-loop* scheme where routine cases are automated while ambiguous ones are reserved for operator review.

The findings confirm the technical, economic, and operational feasibility of intelligent ticket registration automation in medium-sized organizations. The thesis contributes a reproducible architecture on mature open-source components, empirical evidence on large language model performance for ticket classification in Rioplatense Spanish, and a regulatory framework aligned with Argentina's Personal Data Protection Law 25.326.

Keywords: automation, help desk, natural language processing, large language model, N8N, FastAPI, ticket classification, Rioplatense Spanish, human in the loop

Indice general

1	Introduccion	10
1.1	Contextualizacion	10
1.2	Planteamiento del problema	11
1.3	Pregunta de investigacion	12
1.4	Hipotesis de trabajo	12
1.5	Objetivo general	13
1.6	Objetivos especificos	13
1.7	Justificacion	14
1.8	Alcance y delimitaciones	15
2	Marco teorico	17
2.1	Gestion de servicios de tecnologias de la informacion	17
2.2	Mesa de ayuda y modelo ITIL	17
2.3	Automatizacion de procesos y orquestacion de flujos	18
2.4	Procesamiento de lenguaje natural y modelos de lenguaje grandes	19
2.5	Reconocimiento automatico del habla	20
2.6	Bases de datos relacionales y persistencia transaccional	21
2.7	Metricas de evaluacion de clasificadores multiclase	21
3	Estado del arte	23
3.1	Soluciones comerciales de mesa de ayuda	23
3.2	Plataformas de orquestacion de flujos	24
3.3	Trabajos academicos previos sobre clasificacion de tickets	24
3.4	Analisis comparativo y brecha identificada	25
4	Marco metodologico	27
4.1	Enfoque y tipo de investigacion	27
4.2	Diseno experimental	27
4.3	Operacionalizacion de variables	28
4.4	Poblacion, muestra y corpus	28
4.5	Protocolo de pruebas	29
4.6	Metricas e instrumentos	30

4.7	Análisis estadístico	30
4.8	Amenazas a la validez	31
4.9	Gestión del proyecto bajo enfoque Scrumban	31
5	Arquitectura del sistema	33
5.1	Visión general	33
5.2	Capa de canales de entrada	33
5.3	Capa de orquestación.....	34
5.4	Capa de procesamiento	35
5.5	Subsistema de clasificación híbrido	35
5.6	Modelo de datos	36
5.7	Contrato de la interfaz REST	37
6	Implementación	38
6.1	Entorno de despliegue y dependencias	38
6.2	Construcción del módulo Python	38
6.3	Construcción del flujo en N8N	39
6.4	Integración del canal telefónico	41
6.5	Desafíos de integración	41
6.6	Conexión con el proceso Scrumban	42
6.7	Pruebas automatizadas	44
7	Resultados	45
7.1	Comparación de tiempos de registro	45
7.2	Matriz de confusión y métricas de clasificación	46
7.3	Reducción de la intervención humana	47
7.4	Análisis de errores y casos límite	48
8	Discusión	49
8.1	Interpretación de los resultados	49
8.2	Comparación con la literatura	50
8.3	Limitaciones del estudio	51
8.4	Implicancias prácticas.....	52
8.5	Reflexiones sobre la generalización	53

9 Conclusiones	54
9.1 Cumplimiento de los objetivos del trabajo	54
9.2 Aportes y generalizacion	56
10 Recomendaciones y lineas de trabajo futuro	57
10.1 Incorporacion de un clasificador supervisado con corpus propio	57
10.2 Ampliacion de canales de entrada	57
10.3 Panel de monitoreo en tiempo real	57
10.4 Mecanismos de aprendizaje activo	58
10.5 Estudios de replicacion	58
10.6 Evaluacion de modelos de lenguaje de codigo abierto	58
11 Consideraciones eticas y aspectos legales	60
11.1 Marco normativo aplicable	60
11.2 Principios aplicados al tratamiento de datos	60
11.3 Transferencia internacional y pseudonimizacion	61
11.4 Seguridad tecnica	61
11.5 Consentimiento, derechos del usuario y supervision humana	61
Referencias bibliograficas	63
Appendices	66
A Diagrama de arquitectura del sistema	66
B Repositorio de codigo fuente	66
C Esquema de base de datos	66
D Especificacion OpenAPI de la interfaz REST	67
E Configuracion del flujo N8N	67
F Corpus de validacion	68
G Documentacion operativa	68

Indice de tablas

1	Comparacion de soluciones para gestion automatizada de mesa de ayuda	25
2	Operacionalizacion de variables del estudio	28
3	Capas funcionales de la arquitectura del sistema	33
4	Entidades principales del modelo de datos	36
5	Puntos de entrada principales de la interfaz REST	37
6	Estadisticos descriptivos de los tiempos de registro por flujo ($n = 200$)	45
7	Matriz de confusion del clasificador automatico ($n = 200$)	46
8	Metricas de clasificacion por clase y promedio macro	46
9	Tipologia de errores observados en la clasificacion automatica	48

Indice de figuras

1	Diagrama de arquitectura del sistema. La version completa en notacion UML se incluye en el Anexo A.	34
---	--	----

1. Introduccion

1.1. Contextualizacion

La evolucion de las tecnologias de la informacion ha transformado profundamente la dinamica operativa de las organizaciones contemporaneas. En la actualidad los sistemas informaticos constituyen el soporte fundamental para la ejecucion de procesos administrativos, productivos y de comunicacion, y esa dependencia tecnologica ha incrementado de manera correlativa la importancia de los servicios de soporte tecnico, particularmente aquellos relacionados con la gestion de incidentes informaticos. Galup et al. (2009) caracterizan a la disciplina de gestion de servicios de tecnologias de la informacion (ITSM, por su sigla en ingles) como un cuerpo de practicas orientado a alinear la operacion tecnologica con los objetivos del negocio, donde la mesa de ayuda actua como punto unico de contacto entre el usuario y el area de tecnologia.

Dentro de este escenario, las mesas de ayuda empresariales cumplen un rol central en la recepcion, registro y seguimiento de incidentes reportados por los usuarios. Estas unidades organizacionales canalizan solicitudes y constituyen la primera linea de atencion tecnica, de modo que su eficiencia impacta directamente en la continuidad operativa. No obstante, en muchas organizaciones medianas el proceso de registro inicial continua realizandose manualmente, lo que introduce demoras, inconsistencias y una marcada dependencia del criterio individual del operador. Galup et al. (2009) advierten que los procesos manuales de soporte tienden a degradarse cuando el volumen de solicitudes crece de manera no lineal o cuando el personal debe alternar entre tareas operativas heterogeneas.

El contexto argentino presenta características particulares que acentúan esta problemática. Las organizaciones medianas del país, definidas por la Secretaria de la Pequeña y Mediana Empresa como aquellas que emplean entre 50 y 200 trabajadores en el sector servicios, operan con presupuestos de tecnología acotados y enfrentan restricciones cambiarias que encarecen la adopción de soluciones comerciales internacionales. La combinación de estas restricciones con una demanda creciente de servicios digitales configura un escenario en el que las soluciones basadas en componentes de código abierto y servicios de inferencia bajo demanda resultan particularmente atractivas desde el punto de vista económico y operativo.

1.2. Planteamiento del problema

La presente investigacion se desarrolla en el marco de una organizacion mediana del sector servicios con sede en la provincia de Mendoza, Republica Argentina, cuya plantilla asciende a aproximadamente 120 usuarios internos distribuidos en cinco areas funcionales. El sector de Operaciones desempeña simultaneamente tareas vinculadas a la gestion operativa cotidiana y a la atencion de incidentes informaticos reportados por los usuarios, lo que obliga al personal responsable a interrumpir actividades criticas para registrar manualmente fallas de hardware, inconvenientes de software o solicitudes de asistencia tecnica.

Las mediciones preliminares realizadas durante una semana laboral representativa evidenciaron un volumen promedio de 42 incidentes diarios, un tiempo medio de registro manual de 2 minutos con 45 segundos por incidente y una tasa de derivacion erronea inicial cercana al 15 %. La proyeccion trimestral sobre estos indicadores arrojó un universo aproximado de 3.700 incidentes, con un costo operativo asociado a la etapa de registro estimado en el orden de 200 horas-persona por trimestre. Este tiempo, que el personal podría redirigir a tareas de mayor valor agregado si la etapa de registro estuviera automatizada, representa un costo de oportunidad significativo para la organizacion.

El registro manual de incidentes implica además la lectura de correos electronicos, la interpretacion de descripciones textuales heterogeneas, la clasificacion del problema segun el area responsable y la carga manual de la informacion en un sistema de tickets. Este procedimiento requiere tiempo, interrumpe las tareas del personal tecnico y puede generar errores de clasificacion que prolongan la cadena de resolucion cuando un ticket es derivado a un area incorrecta. La dispersion de los canales de comunicacion —correo, llamada telefonica y formularios internos— agrava la situacion al obligar al personal a monitorear multiples fuentes simultaneamente.

La naturaleza del problema presenta un patron recurrente en la literatura de gestion de servicios: un cuello de botella en la etapa inicial del proceso que degrada la eficiencia del ciclo completo. Galup et al. (2009) documentan que los procesos de soporte no automatizados tienden a degradarse cuando el volumen de solicitudes crece, con un impacto desproporcionado en organizaciones medianas donde el personal de soporte combina multiples roles. Estos indicadores justifican la busqueda de una alternativa tecnologica que reduzca el costo operativo, mejore la calidad de la derivacion temprana y disminuya la variabilidad asociada al criterio

individual.

1.3. Pregunta de investigacion

A partir del problema descrito, esta investigacion se orienta a responder la siguiente pregunta: ¿reduce la automatizacion del registro inicial de incidentes mediante orquestacion de flujos y procesamiento de lenguaje natural el tiempo de registro y mejora la exactitud de clasificacion en una mesa de ayuda empresarial mediana, en comparacion con el proceso manual realizado por personal experimentado?

La pregunta se descompone en tres interrogantes subsidiarios que guian el diseno experimental y la seleccion de metricas. Primero, ¿cual es la magnitud de la reduccion del tiempo de registro que puede obtenerse mediante la automatizacion, y resulta esa reduccion estadisticamente significativa? Segundo, ¿alcanza el clasificador automatico una exactitud comparable o superior a la del criterio humano, y como se distribuyen los errores entre las categorias objetivo? Tercero, ¿permite la arquitectura propuesta integrar canales multiples de recepcion sin degradar la latencia ni la calidad de la clasificacion?

1.4. Hipotesis de trabajo

La hipotesis principal de este trabajo sostiene que la implementacion de un sistema automatizado de registro de incidentes basado en orquestacion de flujos con N8N y procesamiento de lenguaje natural mediante un modelo de lenguaje grande permite reducir significativamente el tiempo de registro y la proporcion de intervencion humana, manteniendo una exactitud de clasificacion equivalente o superior a la observada en el proceso manual realizado por personal experimentado.

Formalmente, la hipotesis nula H_0 postula la inexistencia de diferencias significativas entre ambos flujos en las variables observadas: $H_0 : M_{\text{manual}} = M_{\text{automatizado}}$, donde M representa la mediana poblacional del tiempo de registro. La hipotesis alternativa H_1 postula que la mediana del flujo automatizado es inferior: $H_1 : M_{\text{manual}} > M_{\text{automatizado}}$.

Como hipotesis subsidiarias se plantea, en primer lugar, que la integracion de multiples canales de entrada dentro de un unico flujo automatizado mejora la accesibilidad del sistema y reduce la dispersion operativa sin introducir cuellos de botella adicionales. En segundo lugar, que un esquema hibrido de clasificacion basado en reglas deterministicas y modelo de lenguaje grande resulta mas eficiente —en terminos de la relacion entre exactitud, latencia y costo

de inferencia— que un esquema construido exclusivamente sobre modelo externo, dado que una proporcion sustancial de los incidentes presenta marcadores lexicos inequivocos que no requieren consulta al modelo de lenguaje.

1.5. Objetivo general

Desarrollar e implementar un sistema automatizado de mesa de ayuda empresarial basado en orquestacion de flujos con N8N y procesamiento de lenguaje natural en Python que permita recibir, procesar, clasificar y registrar incidentes de usuarios de manera automatica, reduciendo el tiempo de registro y la proporcion de intervencion humana en la etapa inicial del proceso, manteniendo la supervision tecnica para la resolucion de los casos y garantizando la trazabilidad de las decisiones tomadas por el sistema.

1.6. Objetivos especificos

El primer objetivo especifico consiste en disenar una arquitectura distribuida y modular que integre un motor de orquestacion de flujos, un modulo de procesamiento de lenguaje natural, un canal de telefonia con transcripcion automatica del habla y una base de datos relacional, garantizando comunicacion asincronica entre componentes mediante interfaces de programacion de aplicaciones de tipo REST sobre HTTP cifrado.

El segundo objetivo especifico se orienta a implementar un clasificador automatico que asigne cada incidente a uno de los tres sectores responsables —Sistemas, Operaciones o Soporte Tecnico— alcanzando una exactitud global no inferior al 85 % sobre un corpus controlado de 200 casos representativos del entorno operativo, y un valor F1 macro promediado no inferior a 0,85. El umbral del 85 % se fundamenta en tres criterios convergentes. Primero, los trabajos de Paramesh y Shreedhara (2019) reportan exactitudes de aproximadamente 87 % con enfoques de maquinas de vectores de soporte (SVM) y representaciones TF-IDF, estableciendo un piso de referencia para el estado del arte en clasificacion de tickets. Segundo, el costo organizacional de una derivacion erronea implica al menos 10 a 15 minutos adicionales de reasignacion segun las mediciones preliminares de la organizacion, por lo que una exactitud inferior al 85 % resultaria en tiempos de resolucion comparables a los del flujo manual para un volumen de errores inaceptable. Tercero, el margen por encima del 85 % reserva espacio estadistico para que el intervalo de confianza del estimador no solape la zona de rechazo de la hipotesis, dado el tamano muestral de 200 casos.

El tercer objetivo específico plantea integrar tres canales de entrada paralelos —correo electrónico, formulario web y llamada telefónica con transcripción automática— dentro de un único flujo automatizado que normalice la entrada y centralice el procesamiento, mejorando la accesibilidad del sistema sin incrementar la complejidad operativa percibida por los usuarios finales.

El cuarto objetivo específico consiste en evaluar comparativamente el flujo manual y el flujo automatizado en términos de tiempo medio de registro, dispersión, exactitud de clasificación y proporción de intervención humana, mediante un diseño cuasiexperimental con muestras pareadas, prueba estadística no paramétrica y reporte de intervalos de confianza al 95 %. El diseño experimental sigue las recomendaciones de Hernández Sampieri y Mendoza Torres (2018) para investigaciones tecnológicas de carácter aplicado.

El quinto objetivo específico se orienta a documentar exhaustivamente la arquitectura, los procedimientos de despliegue y los resultados experimentales, proponiendo una hoja de ruta de evolución que oriente futuras mejoras y replicaciones del estudio en organizaciones de características comparables.

1.7. Justificación

La justificación del trabajo se sostiene en tres dimensiones complementarias.

Desde la dimensión organizacional, la automatización propuesta libera capacidad operativa del personal técnico, reduce el tiempo de respuesta percibido por los usuarios y disminuye la variabilidad introducida por el criterio individual. La evidencia recogida en este trabajo permite cuantificar esa mejora en el contexto específico de una organización mediana argentina, aportando un caso documentado que otras organizaciones pueden utilizar como referencia para estimar el retorno esperado de una inversión similar.

Desde la dimensión tecnológica, la solución integra herramientas de bajo costo y código abierto —N8N, Python, FastAPI, PostgreSQL— permitiendo una implementación reproducible en organizaciones de tamaño mediano sin requerir grandes inversiones de capital ni dependencia exclusiva de proveedores comerciales. Esta propiedad es particularmente relevante en el contexto argentino, donde las restricciones de acceso a divisas y los costos de suscripción en moneda extranjera constituyen barreras significativas para la adopción de soluciones comerciales internacionales. Además, el despliegue autoalojado de la totalidad de los componentes —con excepción del servicio de inferencia— otorga a la organización control pleno sobre sus

datos operativos, en línea con el principio de soberanía digital promovido por la Ley 25.326 de Protección de los Datos Personales (Honorable Congreso de la Nación Argentina, 2000).

Desde la dimensión académica, el trabajo aporta evidencia empírica sobre el desempeño de modelos de lenguaje grandes aplicados a la clasificación de tickets en español rioplatense, dominio escasamente documentado en la literatura comparativa disponible. La mayoría de los estudios publicados sobre clasificación automática de tickets utilizan corpus en inglés, alemán o portugués, y los pocos trabajos que incluyen el español lo hacen con variantes peninsulares o con corpus genéricos no específicos del dominio de mesas de ayuda. Este trabajo contribuye a cerrar esa brecha mediante la validación rigurosa de un clasificador híbrido sobre un corpus de 200 casos en español rioplatense con doble etiquetado independiente y acuerdo inter-evaluador cuantificado.

Adicionalmente, la mayoría de las soluciones comerciales disponibles —tales como Zendesk Suite, Freshdesk o ServiceNow ITSM— presentan modelos de costos por agente y mes que escalan linealmente con el equipo y operan exclusivamente en modalidad de software como servicio con almacenamiento de datos en jurisdicciones extranjeras. La propuesta desarrollada en esta investigación utiliza herramientas de despliegue autoalojado, lo que facilita su adopción en organizaciones medianas y simplifica el cumplimiento de la normativa argentina de protección de datos personales.

1.8. Alcance y delimitaciones

El alcance del trabajo se circunscribe a la etapa de recepción, clasificación y registro inicial de incidentes, sin abarcar la resolución técnica de los mismos, la cual permanece bajo responsabilidad del personal humano. El sistema contempla tres canales de entrada (correo electrónico, formulario web y llamada telefónica con transcripción automática) y tres sectores de derivación (Sistemas, Operaciones y Soporte Técnico). La validación experimental se realiza sobre un corpus en idioma español rioplatense recolectado en una única organización del sector servicios.

Quedan fuera del alcance los siguientes aspectos, los cuales se proponen como líneas de trabajo futuro en el Capítulo 10. Primero, la integración con sistemas externos de inventario de activos informáticos o con sistemas de gestión de configuración. Segundo, los modelos de aprendizaje supervisado entrenados con corpus propios de la organización, dado que el volumen actual de datos etiquetados resulta insuficiente para esa estrategia. Tercero, la imple-

mentacion de paneles de monitoreo avanzados con visualizacion en tiempo real de metricas operativas. Cuarto, la automatizacion de la etapa de resolucion tecnica, que trasciende el alcance del presente trabajo.

Las delimitaciones del estudio incluyen: la restriccion geografica a una unica organizacion de la provincia de Mendoza, lo que limita la validez externa de los hallazgos; la dependencia operativa del servicio externo de inferencia Gemini 2.5 Flash, cuyo rendimiento y condiciones comerciales escapan al control de los investigadores; y la naturaleza temporal de los resultados, que reflejan el comportamiento del modelo en la version vigente al momento del estudio (junio de 2026), sujeta a modificaciones futuras por parte del proveedor.

2. Marco teorico

Este capitulo establece los fundamentos conceptuales sobre los cuales se construye la investigacion. Se abordan siete ejes tematicos que proporcionan el andamiaje teorico necesario para comprender el diseno, la implementacion y la evaluacion del sistema propuesto.

2.1. Gestion de servicios de tecnologias de la informacion

La gestion de servicios de tecnologias de la informacion (ITSM) constituye una disciplina sistematizada que articula procesos, personas y tecnologias con el proposito de entregar valor al negocio a traves de servicios tecnologicos confiables y mensurables. Galup et al. (2009) caracterizan la disciplina como un cuerpo de practicas orientado a alinear la operacion tecnologica con los objetivos del negocio, donde la mesa de ayuda opera como punto unico de contacto entre el usuario y el area de tecnologia. Este enfoque rompe con la concepcion anterior que entendia a la informatica como un area de soporte estrictamente reactivo y la posiciona, en cambio, como un proveedor de servicios sujeto a acuerdos de nivel de servicio explicitos.

Dentro de este marco general, la gestion de incidentes constituye uno de los procesos operativos centrales. Su objetivo principal consiste en restaurar el servicio afectado en el menor tiempo posible, minimizando el impacto adverso en la operacion del negocio. El proceso comprende cuatro etapas secuenciales: la recepcion y registro inicial, la clasificacion y priorizacion, la asignacion al area responsable y, finalmente, la resolucion y cierre. Galup et al. (2009) advierten que los procesos manuales de soporte tienden a degradarse cuando el volumen de solicitudes crece de manera no lineal o cuando el personal debe alternar entre tareas operativas heterogeneas, situacion que coincide con la observada en el entorno empresarial analizado en el presente trabajo.

2.2. Mesa de ayuda y modelo ITIL

El marco de buenas practicas ITIL (Information Technology Infrastructure Library), en su edicion de operaciones de servicio (Office of Government Commerce, 2011), establece la distincion conceptual entre incidente —definido como una interrupcion no planificada del servicio o una reduccion de su calidad— y solicitud de servicio —definida como un pedido formal de informacion, asesoramiento, acceso a un componente o ejecucion de una operacion estandarizada—. Esta distincion resulta especialmente relevante para el diseno del clasificador

propuesto en este trabajo, ya que orienta la categorizacion inicial y condiciona la priorizacion subsiguiente.

El marco ITIL postula adicionalmente tres principios operativos que guian el diseno de la solucion propuesta. El primero es la centralizacion de la recepcion mediante un punto unico de contacto (SPOC, por su sigla en ingles), que en el sistema propuesto se materializa a traves del flujo unificado de orquestacion que normaliza las entradas de los tres canales. El segundo es la operacion bajo acuerdos de nivel de servicio (SLA), que cuantifican la calidad esperada en terminos de tiempo de respuesta y tiempo de resolucio. El tercero es la mejora continua del proceso, que en el sistema propuesto se habilita mediante la trazabilidad de cada decision del clasificador y la retroalimentacion de los sectores responsables.

Los modelos tradicionales de mesa de ayuda se basan en la interaccion manual entre el usuario y el operador, lo que implica tiempos de respuesta variables y una dependencia directa del personal disponible. Cuando el volumen de incidentes excede la capacidad del equipo o cuando los operadores deben alternar simultaneamente entre tareas operativas y de soporte, la calidad del servicio se degrada de manera observable. La automatizacion del registro inicial constituye, en consecuencia, una intervencion que aborda el cuello de botella en el punto de mayor sensibilidad del proceso, sin requerir la reestructuracion completa del modelo operativo.

2.3. Automatizacion de procesos y orquestacion de flujos

La automatizacion de procesos empresariales (BPA, por su sigla en ingles) permite reducir la intervencion humana en tareas repetitivas mediante la ejecucion automatica de acciones definidas a partir de eventos especificos. Russell y Norvig (2021) ubican a la automatizacion inteligente dentro del paradigma de los agentes orientados a objetivos, en el cual un sistema reactivo percibe eventos del entorno, decide acciones segun una politica preestablecida y actua modificando el estado del entorno. Esta caracterizacion ofrece un marco conceptual robusto para el diseno de sistemas que combinan reglas deterministicas con decisiones derivadas de modelos probabilisticos, como el clasificador hibrido propuesto en este trabajo.

Las herramientas de orquestacion, tambien denominadas plataformas de integracion como servicio (iPaaS), posibilitan integrar distintos servicios, procesar informacion y ejecutar acciones automaticas en funcion de condiciones predefinidas. Estas plataformas se caracterizan por ofrecer modelos visuales de construccion de flujos, soporte nativo para webhooks y disparadores reactivos, y capacidad de transformacion de datos entre sistemas heterogeneos.

n8n GmbH (2024) documenta las capacidades de N8N, la herramienta seleccionada para este trabajo, que se distingue dentro del segmento por su modelo de licencia de código fuente disponible y su capacidad de despliegue completamente autoalojado, característica que resulta decisiva en escenarios donde el tratamiento de datos sensibles exige mantener la información dentro del perímetro organizacional.

El paradigma de orquestación se diferencia conceptualmente del paradigma de coreografía. Hohpe y Woolf (2003) establecen que la orquestación centraliza la lógica de control en un componente único, lo que simplifica la trazabilidad de cada ejecución pero introduce un punto de coordinación cuya disponibilidad condiciona al sistema completo. La coreografía, en cambio, distribuye la coordinación entre los participantes mediante eventos publicados en un bus común, lo que aumenta la robustez frente a fallos parciales pero dificulta la observabilidad. El presente trabajo adopta el enfoque de orquestación por simplicidad operativa y por la prioridad otorgada a la trazabilidad auditiva de cada decisión del sistema, en línea con los requisitos de la Ley 25.326 (Honorable Congreso de la Nación Argentina, 2000).

2.4. Procesamiento de lenguaje natural y modelos de lenguaje grandes

El procesamiento de lenguaje natural (NLP) constituye un área de la inteligencia artificial orientada a la interpretación del lenguaje humano por parte de sistemas informáticos. Jurafsky y Martin (2023) sistematizan la disciplina en torno a tres ejes principales: el modelado estadístico de secuencias, la representación distribuida de palabras y la comprensión semántica mediante arquitecturas neuronales profundas. Mediante estas técnicas resulta posible analizar texto o audio transcrito, identificar palabras clave, determinar la intención del usuario y clasificar contenido en categorías predefinidas.

La evolución del campo durante la última década ha estado dominada por la arquitectura Transformer (Vaswani et al. 2017), cuyo mecanismo de atención permite a los modelos capturar dependencias de largo alcance sin las limitaciones de las redes neuronales recurrentes. El mecanismo de atención multi-cabeza, en particular, permite que el modelo pondere simultáneamente distintas partes de la secuencia de entrada, lo que resulta especialmente útil para la clasificación de descripciones textuales donde la información relevante puede aparecer en cualquier posición del texto.

Sobre esta arquitectura se construyeron los modelos de lenguaje grandes (LLM) contemporáneos, los cuales exhiben capacidades robustas de clasificación en escenarios de po-

cos ejemplos (*few-shot*) o sin ejemplos (*zero-shot*), fenomeno conocido como aprendizaje en contexto. Brown et al. (2020) demostraron que modelos suficientemente grandes pueden generalizar a tareas nuevas a partir de instrucciones expresadas en lenguaje natural, sin requerir ajuste fino sobre datos especificos del dominio. Esta capacidad es particularmente relevante para el contexto de esta investigacion, donde el volumen de datos etiquetados resulta insuficiente para entrenar un clasificador supervisado tradicional desde cero.

En el contexto de una mesa de ayuda, el uso de NLP permite clasificar incidentes y derivarlos al sector correspondiente sin intervencion manual, mejorando la velocidad de respuesta y reduciendo la carga operativa del personal encargado del registro. La eleccion entre un clasificador supervisado entrenado sobre corpus especifico, un modelo de lenguaje grande de proposito general invocado mediante instrucciones, o un esquema hibrido que combine ambas estrategias, constituye una decision de diseno que depende de la disponibilidad de datos etiquetados, del presupuesto de inferencia disponible y de los requerimientos de soberania sobre los datos procesados.

2.5. Reconocimiento automatico del habla

Los sistemas de reconocimiento automatico del habla (ASR) permiten convertir audio en texto mediante modelos previamente entrenados sobre grandes corpus de habla anotada. Rabiner y Juang (1993) describen los fundamentos clasicos del reconocimiento basado en modelos ocultos de Markov (HMM), que durante decadas constituyeron el enfoque dominante. La generacion contemporanea de sistemas se apoya mayoritariamente en redes neuronales profundas con arquitecturas codificador-decodificador, de las cuales el modelo Whisper (Radford et al. 2023) representa un exponente reciente con alta robustez en escenarios multilingues y en presencia de ruido ambiental.

Whisper fue entrenado sobre 680.000 horas de audio multilingue recolectadas de la web, lo que le confiere una capacidad de generalizacion notablemente superior a la de sistemas entrenados sobre corpus curados de menor escala. Su arquitectura codificador-decodificador con atencion procesa el audio en segmentos de 30 segundos y genera transcripciones que incluyen puntuacion y normalizacion ortografica. La disponibilidad de modelos preentrenados de codigo abierto para espanol facilita su integracion en el canal telefonico del sistema propuesto.

En el contexto de la mesa de ayuda, la integracion entre reconocimiento del habla y procesamiento de lenguaje natural permite automatizar completamente la recepcion y clasifi-

cacion de solicitudes telefonicas, posibilitando que el sistema interprete la solicitud del usuario y genere el ticket correspondiente sin intervencion humana en la etapa de captura de voz.

2.6. Bases de datos relacionales y persistencia transaccional

La persistencia de los incidentes en una base de datos relacional permite garantizar las propiedades de atomicidad, consistencia, aislamiento y durabilidad (ACID) propias del modelo transaccional clasico (Date, 2003). PostgreSQL, el motor seleccionado para este trabajo, ofrece soporte completo del estandar SQL, extensiones para tipos de datos avanzados como JSON binario y arreglos, y un modelo de concurrencia basado en control multiversional (MVCC) que resulta adecuado para cargas mixtas de lectura y escritura (The PostgreSQL Global Development Group, 2024).

La eleccion de un motor relacional frente a alternativas no relacionales se sustenta en la naturaleza estructurada de los datos del dominio, donde las relaciones entre incidentes, sectores, estados y canales de origen se modelan naturalmente como tablas vinculadas mediante claves foraneas con integridad referencial declarativa. Esta modelizacion relacional facilita ademas la auditoria y la trazabilidad de las operaciones, en linea con los requisitos normativos identificados en la seccion de consideraciones eticas y legales (Capitulo 11).

2.7. Metricas de evaluacion de clasificadores multiclase

La evaluacion de clasificadores multiclase requiere metricas que capturen tanto el desempeno global como el desempeno por clase. Powers (2011) describe el conjunto canonico de metricas derivadas de la matriz de confusion, entre las cuales destacan la exactitud global (*accuracy*), la precision por clase, la sensibilidad o exhaustividad (*recall*) por clase, y la medida F1 como media armonica de las dos anteriores.

La exactitud global se define como:

$$\text{Exactitud} = \frac{TP + TN}{TP + TN + FP + FN}$$

donde TP, TN, FP y FN representan verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos, respectivamente. Para problemas de clasificacion multiclase, la exactitud se calcula como la proporcion de predicciones correctas sobre el total de predicciones.

Sokolova y Lapalme (2009) advierten que en escenarios con clases desbalanceadas la exactitud global puede resultar enganosa y recomiendan reportar valores macro promediados,

los cuales otorgan igual peso a cada clase con independencia de su frecuencia. El valor F1 macro promediado se define como:

$$F1_{\text{macro}} = \frac{1}{k} \sum_{i=1}^k F1_i$$

donde k es el numero de clases y $F1_i$ es el valor F1 de la clase i . El presente trabajo adopta esta recomendacion y reporta tanto exactitud global como F1 macro y metricas desagregadas por clase, permitiendo al lector evaluar la robustez del clasificador frente a la distribucion especifica del corpus.

Adicionalmente, la matriz de confusion constituye el instrumento estandar de presentacion de resultados en clasificacion multiclase, ya que permite identificar patrones especificos de error: confusiones sistematicas entre dos categorias particulares, sesgos hacia una clase mayoritaria o aciertos preferenciales sobre una clase especifica. La inspeccion de la matriz orienta el analisis cualitativo posterior y constituye la base para cualquier propuesta de mejora del clasificador.

3. Estado del arte

Este capítulo revisa la literatura académica y las soluciones comerciales disponibles en tres áreas relevantes para el presente trabajo: plataformas de mesa de ayuda, herramientas de orquestación de flujos, y métodos de clasificación automática de tickets de soporte. El propósito es situar la contribución de esta investigación en el contexto de los desarrollos previos y fundamentar la brecha que motiva el trabajo.

3.1. Soluciones comerciales de mesa de ayuda

El segmento de soluciones comerciales para gestión de mesa de ayuda se encuentra dominado por plataformas integrales que cubren el ciclo completo de vida del ticket. Zendesk Suite ofrece funcionalidades de recepción multicanal, automatización de flujos básicos y, mediante un módulo adicional, capacidades de inteligencia artificial para sugerencia de respuestas y deflexión de consultas frecuentes. Freshdesk propone una arquitectura comparable e incorpora el motor Freddy AI para clasificación automática y enrutamiento inteligente. Jira Service Management, parte del ecosistema de Atlassian, ofrece capacidades limitadas de automatización inteligente en su plan estándar; la edición Data Center permite el despliegue autoalojado para organizaciones con requisitos de soberanía sobre los datos. ServiceNow ITSM constituye la opción de referencia en el segmento corporativo, con un módulo Predictive Intelligence que aplica aprendizaje automático sobre el historico de tickets de la organización.

La adopción de capacidades de inteligencia artificial en plataformas comerciales de mesa de ayuda ha crecido de manera sostenida en los últimos años, particularmente en organizaciones medianas de mercados de habla inglesa. No obstante, estas plataformas presentan dos limitaciones relevantes para el contexto de la presente investigación. La primera es el modelo de costos basado en agente y mes, que escala linealmente con el tamaño del equipo de soporte. La segunda es que la mayoría de estas plataformas operan exclusivamente en modalidad de software como servicio (SaaS) con almacenamiento de datos en jurisdicciones extranjeras, lo que introduce consideraciones adicionales sobre transferencia internacional de datos personales bajo la Ley 25.326 (Honorable Congreso de la Nación Argentina, 2000).

En el segmento de código abierto, osTicket constituye una alternativa de despliegue autoalojado sin costo de licencia, aunque carece de capacidades nativas de clasificación automática mediante procesamiento de lenguaje natural moderno. Esta brecha funcional motiva la búsqueda de arquitecturas compositivas que integren componentes maduros de código abierto

con servicios de inferencia bajo demanda.

3.2. Plataformas de orquestacion de flujos

Dentro del segmento de plataformas de orquestacion de flujos, las soluciones mas difundidas incluyen Zapier, Make (anteriormente Integromat), Microsoft Power Automate, Apache Airflow y N8N. Las primeras tres operan exclusivamente como SaaS, lo que las descarta como base para implementaciones donde el control sobre la infraestructura es un requisito explicito. Apache Airflow se orienta a flujos de trabajo programados de tipo procesamiento de datos por lotes, con un diseno centrado en grafos aciclicos dirigidos (DAG) definidos como codigo Python (Apache Software Foundation, 2024); aunque potente, su paradigma de ejecucion programada resulta inadecuado para un caso de uso reactivo y orientado a eventos como el de una mesa de ayuda.

N8N ofrece un modelo de construccion visual de flujos sobre nodos predefinidos, soporte nativo para webhooks y disparadores reactivos, ejecucion basada en eventos y posibilidad de despliegue completamente autoalojado bajo Docker (n8n GmbH, 2024). La disponibilidad de su codigo fuente bajo licencia Sustainable Use facilita la auditoria de seguridad y la adaptacion a requerimientos especificos sin generar dependencia exclusiva del proveedor. Estas características confluyen para posicionar a N8N como la opcion mas adecuada para el escenario propuesto, donde la soberania sobre los datos y los flujos de procesamiento constituye un requisito no negociable.

3.3. Trabajos academicos previos sobre clasificacion de tickets

La literatura academica sobre clasificacion automatica de tickets de soporte ha experimentado un crecimiento considerable durante los ultimos anos. A continuacion se presentan los trabajos mas relevantes, organizados por enfoque metodologico y por orden cronologico.

Paramesh y Shreedhara (2019) constituyen uno de los primeros trabajos exhaustivos en el area. Utilizando combinaciones de maquinas de vectores de soporte (SVM) y representaciones TF-IDF sobre un corpus en ingles de aproximadamente 10.000 tickets, reportan exactitudes cercanas al 87 %. Su contribucion principal radica en la demostracion de que las tecnicas classicas de aprendizaje supervisado, cuando se aplican sobre representaciones textuales adecuadas, pueden alcanzar niveles de desempeno operativamente utiles para la clasificacion automatica de tickets.

Revina et al. (2020) extienden este enfoque incorporando arquitecturas basadas en BERT y reportan mejoras significativas, alcanzando exactitudes del orden del 91 % sobre un corpus de tamaño comparable. Su hallazgo más relevante para el presente trabajo es la constatación de que modelos preentrenados sobre grandes corpus de texto general pueden transferir conocimiento lingüístico útil al dominio específico de los tickets de soporte, incluso sin ajuste fino sobre datos del dominio. Este resultado respalda la estrategia adoptada en la presente investigación de utilizar un LLM de propósito general como motor de inferencia.

La literatura reciente sobre evaluación de modelos de lenguaje en escenarios de pocos ejemplos ha documentado que el rendimiento de clasificación depende fuertemente del idioma del corpus, con resultados que favorecen a los idiomas mejor representados en los corpus de entrenamiento de los modelos contemporáneos. El español, y en particular sus variantes regionales como el rioplatense, ha recibido una atención notablemente menor en las evaluaciones sistemáticas publicadas. Esta brecha constituye una de las motivaciones académicas del presente trabajo, el cual aporta evidencia empírica sobre el desempeño del clasificador en español rioplatense aplicado a un dominio de mesa de ayuda de organización mediana.

3.4. Análisis comparativo y brecha identificada

La Tabla 1 presenta la comparación sistemática de las principales soluciones revisadas frente a la propuesta del presente trabajo, considerando cuatro dimensiones relevantes: tipo de despliegue, capacidad de clasificación automática, soberanía sobre los datos y modelo de costos.

Tabla 1. Comparación de soluciones para gestión automatizada de mesa de ayuda

Solución	Despliegue	Clasificación automática	Modelo de costo
Zendesk Suite	SaaS comercial	Si, módulo de IA	Alto, por agente/mes
Freshdesk	SaaS comercial	Si, motor Freddy AI	Medio-alto, por agente
Jira Service Mgmt	SaaS o autoalojado	Limitada en plan estándar	Medio, por agente
ServiceNow ITSM	SaaS comercial	Si, Predictive Intelligence	Muy alto, contrato anual
osTicket	Código abierto	No nativa	Sin costo
Propuesta	Híbrida (iPaaS + módulo propio)	Si, LLM + reglas deterministas	Bajo, costo por uso de API

La síntesis comparativa evidencia que ninguna de las soluciones revisadas combina simultáneamente las cuatro propiedades que el contexto del trabajo demanda: despliegue autoalojado para resguardo de datos sensibles, costo bajo y predecible en el tiempo, capacidad nativa de clasificación automática mediante NLP moderno, y soporte específico para canales múltiples incluyendo telefonía con transcripción automática. La brecha identificada se sintetiza en la ausencia de soluciones de bajo costo, autoalojadas, con clasificación basada en modelos de lenguaje grandes y validadas empíricamente sobre corpus en español rioplatense para el dominio específico de mesas de ayuda empresariales medianas.

La presente investigación se ubica precisamente en esa intersección y propone una arquitectura compositiva que reutiliza componentes maduros de código abierto y servicios de inferencia bajo demanda, manteniendo bajo control de la organización los datos sensibles y los flujos de orquestación, y aportando evidencia empírica reproducible sobre el desempeño del enfoque en un contexto lingüístico y organizacional escasamente documentado en la literatura.

4. Marco metodológico

4.1. Enfoque y tipo de investigacion

La investigacion se enmarca dentro del enfoque cuantitativo aplicado, dado que propone el desarrollo de una solucion informatica orientada a mejorar el funcionamiento de un proceso organizacional concreto y mide su impacto mediante indicadores numericos. Hernández Sampieri y Mendoza Torres (2018) ubican este tipo de estudios dentro de la investigacion tecnologica de caracter aplicado, donde el conocimiento producido se orienta a la transformacion practica de la realidad antes que a la generacion de teoria general. El trabajo posee adicionalmente caracter experimental, en la medida en que disena, implementa y evalua un artefacto tecnologico bajo condiciones controladas.

4.2. Diseno experimental

El diseno experimental adoptado es de tipo cuasiexperimental con muestras pareadas y mediciones repetidas. La unidad de observacion es el incidente individual, y cada uno se procesa secuencialmente bajo dos condiciones: la condicion manual y la condicion automatizada. Esta estrategia —procesar el mismo conjunto de incidentes por ambas vias— reduce sustancialmente la variabilidad asociada a las caracteristicas intrinsecas de cada caso, lo que aumenta la potencia estadistica del contraste entre flujos.

Para mitigar el efecto de aprendizaje del operador humano sobre el corpus, se aplico contrabalanceo del orden de procesamiento: el corpus de 200 casos se dividio en dos mitades equilibradas de 100 incidentes cada una, procesandose la primera mitad por la via manual seguida de la via automatizada, y la segunda mitad en orden inverso. Este contrabalanceo distribuye simetricamente cualquier posible ventaja derivada del orden de exposicion al corpus.

La investigacion se desarrollo bajo un protocolo de observacion naturalista donde los operadores no fueron informados de que sus tiempos estaban siendo registrados como parte de un estudio, sino unicamente de que se estaba utilizando el nuevo sistema para procesamiento de incidentes. Este enfoque elimina el efecto Hawthorne —la alteracion del comportamiento de los sujetos cuando saben que estan siendo observados— al mantener la condicion operativa como indistinguible de la practica laboral regular (Hernández Sampieri & Mendoza Torres, 2018). La medicion del tiempo se realizo mediante registros automaticos del propio sistema en el caso del flujo automatizado (marcas de tiempo de entrada y salida en la base de

datos, con precision al milisegundo) y mediante registros administrativos de sistemas de soporte posteriores al ciclo de pruebas en el caso del flujo manual, evitando cronometraje visual explicito que pudiera alertar a los operadores.

4.3. Operacionalizacion de variables

La Tabla 2 presenta la operacionalizacion de las variables del estudio, distinguiendo entre variables cuantitativas continuas, cuantitativas proporcionales y cualitativas nominales. Cada variable se acompaña de su unidad de medida y de la definicion operacional aplicada durante la captura, garantizando la trazabilidad entre la definicion conceptual y el procedimiento de medicion efectivo.

Tabla 2. Operacionalizacion de variables del estudio

Variable	Tipo	Unidad	Definicion operacional
Tiempo de registro	Cuantitativa continua	Segundos (s)	Lapso entre la recepcion del incidente en el canal de entrada y la persistencia confirmada del ticket en la base de datos.
Exactitud de clasificacion	Cuantitativa proporcional	Porcentaje [0; 100]	Cociente entre incidentes correctamente clasificados y total de incidentes evaluados.
F1 macro promediado	Cuantitativa proporcional	Adimensional [0; 1]	Promedio aritmetico del valor F1 calculado de forma independiente para cada una de las tres clases objetivo.
Intervencion humana	Cuantitativa proporcional	Porcentaje [0; 100]	Proporcion del tiempo total del proceso que requiere accion manual de un operador humano.
Canal de origen	Cualitativa nominal	Categoria	Via de entrada: correo electronico, formulario web o llamada telefonica.
Categoria asignada	Cualitativa nominal	Categoria	Sector responsable: Sistemas, Operaciones o Soporte Tecnico.

4.4. Poblacion, muestra y corpus

La poblacion de referencia esta constituida por la totalidad de incidentes informaticos reportados a la mesa de ayuda de la organizacion analizada durante un trimestre representativo (julio a septiembre de 2025). A partir de un universo aproximado de 3.700 incidentes

registrados durante ese periodo, se construyo por muestreo estratificado proporcional un corpus de 200 casos distribuidos entre las tres categorias objetivo en la misma proporcion que la poblacion original: 82 casos para Sistemas, 64 casos para Operaciones y 54 casos para Soporte Tecnico.

El tamano muestral se determino con un nivel de confianza del 95 % y un margen de error del 6,5 %, valores razonables para un estudio piloto orientado a evaluar viabilidad tecnica antes de un despliegue definitivo. El calculo siguio la formula clasica para muestreo aleatorio simple en poblaciones finitas, con correccion por finitud aplicada al universo trimestral observado.

Cada incidente del corpus fue revisado y etiquetado de forma independiente por dos analistas con experiencia en la mesa de ayuda, y las discrepancias fueron resueltas por consenso bajo la supervision del director del trabajo. El acuerdo entre evaluadores, calculado mediante el coeficiente Kappa de Cohen (Cohen, 1960), alcanzo un valor de $\kappa = 0,87$, considerado sustancial segun los rangos de interpretacion habitualmente aceptados ($\kappa > 0,80$ indica acuerdo casi perfecto). Este valor respalda la confiabilidad de la categoria de referencia utilizada como verdad fundamental (*ground truth*) para evaluar el clasificador automatico.

4.5. Protocolo de pruebas

El protocolo de pruebas se ejecuto en dos fases sucesivas. La primera fase consistio en una validacion funcional del sistema, en la cual se verifico la correcta integracion de los componentes mediante: (a) pruebas unitarias del modulo Python con cobertura superior al 80 %, (b) pruebas de integracion del flujo N8N orientadas a verificar el comportamiento extremo a extremo de cada canal, y (c) pruebas de aceptacion simuladas con incidentes sinteticos representativos de cada categoria.

La segunda fase consistio en la validacion experimental propiamente dicha, en la cual los 200 incidentes del corpus se procesaron bajo ambas condiciones operativas y se registraron las variables descritas en la Seccion 4.3. El procesamiento manual fue realizado por dos operadores experimentados con mas de tres anos de antiguedad en la mesa de ayuda, ambos capacitados previamente en el uso del sistema de gestion de tickets utilizado como referencia. El procesamiento automatizado se ejecuto sobre la misma infraestructura, sin intervencion humana, durante una ventana operativa controlada de tres dias habiles consecutivos y con monitoreo continuo de los tiempos de respuesta.

4.6. Metricas e instrumentos

La evaluacion del sistema se sustenta en cuatro metricas principales. La primera es el tiempo medio de registro, calculado como la media aritmetica y la mediana de los tiempos individuales bajo cada condicion, junto con su desvio estandar e intervalo de confianza al 95 %. La segunda es la exactitud global de clasificacion, calculada como la proporcion de incidentes correctamente derivados al sector responsable. La tercera es el valor F1 macro promediado, calculado como la media aritmetica de los valores F1 obtenidos por cada clase, con el fin de neutralizar el efecto del desbalanceo de clases observado en el corpus (Sokolova & Lapalme, 2009). La cuarta es la proporcion de intervencion humana, calculada como el cociente entre el tiempo dedicado a tareas manuales y el tiempo total del proceso.

Como instrumentos de captura se utilizaron: (a) registros automaticos generados por el propio sistema con marcas de tiempo precisas al milisegundo para el flujo automatizado, (b) planillas de cronometraje con medicion observacional sin retroalimentacion inmediata para el flujo manual, y (c) matrices de confusion generadas mediante la biblioteca scikit-learn (Pedregosa et al. 2011).

4.7. Analisis estadistico

Para la comparacion de los tiempos de registro entre ambas condiciones se aplico la prueba de Wilcoxon de rangos con signo, prueba no parametrica para muestras pareadas, dado que la prueba previa de normalidad de Shapiro-Wilk rechazo la hipotesis de normalidad en ambos grupos con valores $p < 0,01$. El nivel de significancia se fijo en $\alpha = 0,05$.

Para cuantificar la magnitud practica de la diferencia se calculo el coeficiente r de correlacion rank-biserial, indicador de tamano del efecto para pruebas no parametricas cuyo rango oscila entre -1 y $+1$. El calculo sigue la formula:

$$r = 1 - \frac{2W}{n(n+1)}$$

donde W es el estadistico de Wilcoxon y n el numero de pares.

Para el analisis de la clasificacion se construyo la matriz de confusion global y se calcularon los valores de precision, sensibilidad (recall) y F1 por cada clase, asi como su promedio macro. Los intervalos de confianza para las proporciones se estimaron mediante el metodo de Wilson, recomendado para muestras moderadas (Wilson, 1927). Todos los calculos se reali-

zaron mediante la biblioteca SciPy del ecosistema Python (Virtanen et al. 2020).

4.8. Amenazas a la validez

Se identificaron y trataron explícitamente cuatro amenazas a la validez del estudio.

La primera, una amenaza a la validez interna por efecto de aprendizaje del operador humano sobre el corpus, se mitigó mediante el contrabalanceo del orden de procesamiento descrito en la Sección 4.2 y mediante la rotación de operadores entre las dos mitades del corpus.

La segunda, una amenaza a la validez de constructo por la subjetividad inherente a la categorización de los incidentes, se mitigó mediante el doble etiquetado independiente, el cálculo del coeficiente Kappa de Cohen y la resolución de discrepancias por consenso supervisado. El valor obtenido ($\kappa = 0,87$) respalda la solidez del constructo de medición.

La tercera, una amenaza a la validez externa derivada del idioma y del dominio organizacional, se reconoce como limitación explícita del estudio: los resultados no son extrapolables sin nuevas validaciones a corpus en otros idiomas, en otras organizaciones o en otros sectores de la economía.

La cuarta, una amenaza a la validez de conclusión derivada del tamaño moderado de la muestra ($n = 200$), se trató mediante el reporte de intervalos de confianza y la elección de pruebas no paramétricas robustas frente a tamaños muestrales pequeños y a desviaciones de la normalidad (Hernández Sampieri & Mendoza Torres, 2018).

4.9. Gestión del proyecto bajo enfoque Scrumban

La construcción del sistema se organizó bajo un esquema Scrumban, combinando la planificación por iteraciones propia de Scrum con la gestión continua del flujo de trabajo característica del método Kanban (Ladas, 2009). Se ejecutaron seis iteraciones de dos semanas cada una, totalizando 12 semanas de desarrollo efectivo:

Iteración 1: Elicitación de requisitos y diseño preliminar de la arquitectura.

Iteración 2: Construcción del módulo Python: modelo de datos, repositorios y servicios.

Iteración 3: Configuración inicial del flujo N8N y primeras integraciones.

Iteración 4: Integración del canal de telefonía y persistencia en PostgreSQL.

Iteración 5: Validación funcional y ajuste fino del clasificador híbrido.

Iteración 6: Validación experimental y documentación final del sistema.

La distribucion de roles asigno la coordinacion general y la integracion de componentes a Luca Gomez, el desarrollo del modulo Python y el modelado de la base de datos a Carla Bustos, y la configuracion del flujo N8N junto con la integracion telefonica mediante Twilio a Gonzalo Sevilla, bajo supervision academica continua del director del trabajo, Prof. Alberto Cortez.

El tablero Kanban se gestiono mediante GitHub Projects, con seguimiento diario del estado de las tareas y reuniones de planificacion al inicio de cada iteracion. Las definiciones de terminado (*Definition of Done*) incluyeron, para cada tarea, la aprobacion de las pruebas unitarias correspondientes, la revision de codigo por pares y la actualizacion de la documentacion asociada.

5. Arquitectura del sistema

5.1. Vision general

El sistema propuesto se estructura como una arquitectura distribuida y modular compuesta por cinco capas claramente diferenciadas, siguiendo los principios de cohesion alta y acoplamiento bajo descritos por Pressman y Maxim (2020). La comunicacion entre capas se realiza mediante el protocolo HTTP sobre TLS 1.3, utilizando interfaces de programacion de aplicaciones de tipo REST con cuerpos JSON para el intercambio estructurado de datos (Fielding & Taylor, 2002). Esta separacion de responsabilidades facilita el reemplazo independiente de cualquier componente sin afectar a los demas.

La Tabla 3 sintetiza las cinco capas y sus responsabilidades especificas.

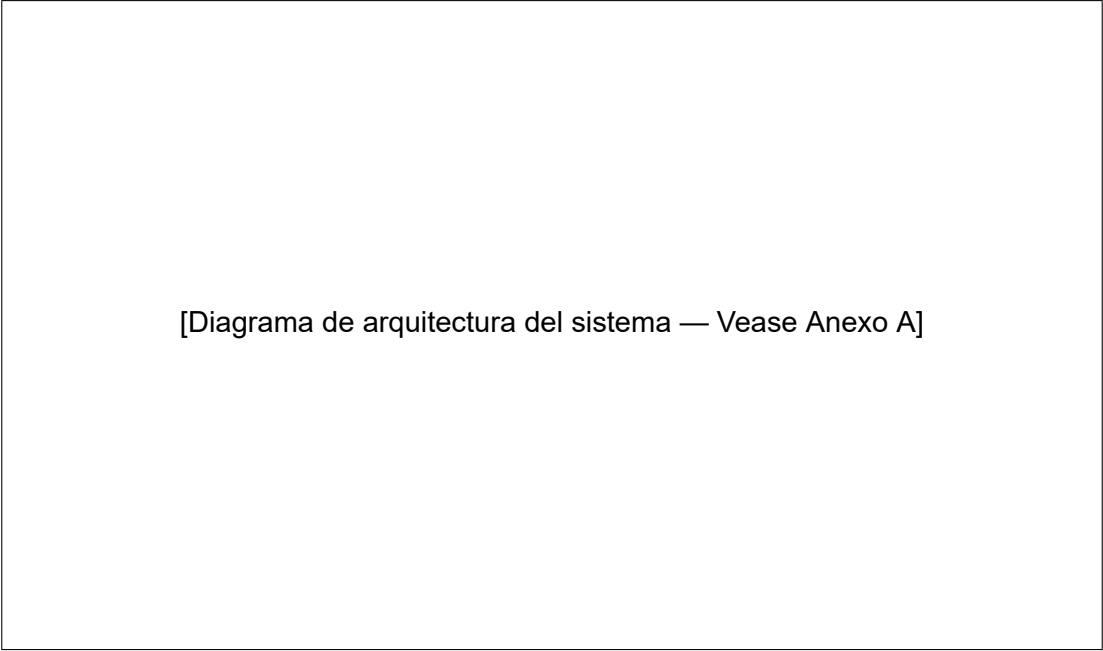
Tabla 3. Capas funcionales de la arquitectura del sistema

Capa	Componentes y responsabilidades
1. Canales de entrada	Casilla IMAP de correo electronico, formulario web expuesto como webhook y numero virtual de Twilio con transcripcion automatica del habla.
2. Orquestracion	Motor N8N v1.62 desplegado en contenedor Docker autoalojado. Normaliza la entrada y coordina la invocacion a las capas inferiores.
3. Procesamiento	Servicio Python sobre FastAPI 0.115 ejecutado bajo Uvicorn. Aplica clasificador hibrido en dos etapas (reglas + LLM) y expone una API REST documentada con OpenAPI 3.1.
4. Inferencia	Modelo Gemini 2.5 Flash de Google, invocado unicamente cuando el filtro deterministico previo no alcanza el umbral de confianza preestablecido.
5. Persistencia	PostgreSQL 15.5 desplegado en contenedor con volumen persistente y respaldos diarios. Almacena los incidentes y la trazabilidad de cada decision del clasificador.

La trazabilidad operativa se garantiza mediante registros estructurados en cada capa, accesibles desde la capa de orquestacion. La Figura 1 representa esquematicamente el flujo de informacion entre capas.

5.2. Capa de canales de entrada

La capa de canales de entrada habilita tres vias paralelas a traves de las cuales un usuario puede reportar un incidente. La primera es el correo electronico, monitoreado por un nodo IMAP de N8N que detecta nuevos mensajes en una casilla institucional dedicada. La



[Diagrama de arquitectura del sistema — Vease Anexo A]

Figura 1. Diagrama de arquitectura del sistema. La version completa en notacion UML se incluye en el Anexo A.

segunda via es un formulario web alojado bajo un punto de entrada webhook expuesto por N8N, accesible desde la intranet corporativa mediante autenticacion corporativa unica (SSO). La tercera via es la llamada telefonica, en la cual el usuario marca un numero virtual provisto por Twilio, recibe instrucciones por voz pregrabadas y deja su solicitud; la grabacion es transcrita automaticamente por el servicio de reconocimiento del habla del proveedor y enviada al flujo N8N mediante webhook posterior al cuelgue. Esta diversidad de canales amplia la accesibilidad del sistema y reduce las barreras al reporte oportuno.

5.3. Capa de orquestacion

N8N actua como orquestador principal del flujo de trabajo. Cada canal de entrada dispara un disparador especifico que normaliza la informacion en una estructura unificada con campos de identificador unico (UUID v4), marca temporal con precision al milisegundo, canal de origen y descripcion textual completa. A continuacion, un nodo HTTP envia esta estructura al modulo de procesamiento Python expuesto como servicio interno, recibe la respuesta con la clasificacion sugerida y la confianza asociada, e invoca finalmente la interfaz REST del sistema de gestion de incidentes para persistir el ticket en PostgreSQL. Los registros de cada ejecucion se conservan durante 30 dias para fines de auditoria y diagnostico operativo.

5.4. Capa de procesamiento

El modulo de procesamiento se implemento como un servicio web ligero utilizando el marco FastAPI 0.115, expuesto sobre un servidor ASGI Uvicorn 0.32 (Tiangolo, 2024). La eleccion de FastAPI obedece a su soporte nativo de validacion de esquemas mediante Pydantic, su rendimiento competitivo y su documentacion automatica conforme a la especificacion OpenAPI 3.1.

Las dependencias del modulo se organizan por capas funcionales claramente delimitadas. La capa de acceso a datos utiliza SQLAlchemy 2.0 como mapeador objeto-relacional (ORM) y psycopg2-binary 2.9 como adaptador de bajo nivel para PostgreSQL. La capa de procesamiento textual emplea las bibliotecas regex y unicodedata de la biblioteca estandar de Python para normalizacion ortografica y eliminacion de signos diacriticos. La capa de inferencia se comunica con la interfaz oficial del modelo Gemini 2.5 Flash mediante el cliente google-generativeai 0.8, con parametros de configuracion optimizados para exactitud y latencia: temperatura = 0,3, top_p = 0,9, max_tokens = 100, timeout = 10 s. Las bibliotecas NumPy 2.1 y Pandas 2.2 se utilizan exclusivamente para la generacion de reportes agregados de desempeno y no participan del flujo de procesamiento individual de incidentes.

5.5. Subsistema de clasificacion hibrido

El subsistema de clasificacion adopta un enfoque hibrido en dos etapas, caracteristica que lo diferencia de aproximaciones puramente basadas en modelo externo. La primera etapa aplica un filtro deterministico basado en expresiones regulares y diccionarios de terminos del dominio. Para mitigar el riesgo de fuga de datos (*data leakage*), la construccion del filtro deterministico se realizo de forma rigurosa: primero, se definieron manualmente un conjunto de patrones regex basados en conocimiento del dominio y analisis preliminar de descripciones de incidentes (analisis que no utilizo el corpus de validacion de 200 casos); segundo, los parametros del prompt enviado a Gemini 2.5 Flash se ajustaron exclusivamente mediante validacion manual sobre incidentes representativos en entornos de preproduccion, sin acceso al corpus de validacion.

Este filtro es capaz de derivar de forma inmediata aquellos incidentes que contienen marcadores inequivocos. Por ejemplo, la presencia simultanea de las palabras `impresora`, `papel` y `atasco` sugiere con alta certeza una categoria de Soporte Tecnico. Cuando este filtro inicial alcanza un umbral de confianza superior al 90 %, la decision se toma sin consultar al

modelo externo, lo que reduce sustancialmente la latencia y el costo de inferencia.

La segunda etapa, activada unicamente cuando el filtro deterministico no alcanza el umbral, consulta al modelo Gemini 2.5 Flash mediante un prompt estructurado en espanol que incluye una breve descripcion de cada categoria, ejemplos representativos balanceados y la solicitud explicita de devolver la categoria asignada y un valor numerico de confianza entre 0 y 1 en formato JSON estricto. La justificacion de este diseno compositivo radica en aprovechar la rapidez del filtro deterministico para los casos triviales y reservar la capacidad semantica del modelo de lenguaje grande para los casos ambiguos o de redaccion atipica, optimizando la relacion entre exactitud, costo y latencia. La medicion sobre el corpus de validacion muestra que aproximadamente el 62 % de los incidentes son resueltos por la primera etapa, mientras que el 38 % restante requiere la consulta al modelo externo.

5.6. Modelo de datos

La persistencia se realiza sobre PostgreSQL 15.5, desplegado en contenedor Docker bajo configuracion de respaldo diario y replicacion fisica opcional para entornos productivos (The PostgreSQL Global Development Group, 2024). El modelo de datos se organiza alrededor de cinco entidades principales, descritas en la Tabla 4.

Tabla 4. Entidades principales del modelo de datos

Tabla	Atributos clave	Descripcion
incidente	id_incidente (PK), id_canal (FK), id_sector (FK), id_estado (FK)	Tabla central: fecha de creacion, descripcion cruda pseudonimizada, prioridad, sector asignado y estado actual.
sector	id_sector (PK), nombre	Catalogo de los tres sectores: Sistemas, Operaciones y Soporte Tecnico.
estado	id_estado (PK), nombre	Catalogo de estados: nuevo, en proceso, en espera, resuelto y cerrado.
canal_origen	id_canal (PK), nombre	Catalogo de canales: correo, formulario web y llamada telefonica.
clasificacion_log	id_log (PK), id_incidente (FK), categoria_predicha, confianza, categoria_validada	Registro de cada decision del clasificador, con categoria predicha, confianza asociada y categoria validada por el sector responsable.

La entidad central `incidente` referencia mediante claves foraneas a las entidades

`sector`, `estado` y `canal_origen`, y mantiene una relacion uno a muchos con `clasificacion_log` para preservar la trazabilidad historica de las decisiones del clasificador. Los identificadores de incidente se generan mediante una secuencia con prefijo configurable, lo cual produce numeros legibles y consistentes que se comunican al usuario al momento del registro. El esquema completo, incluyendo restricciones de integridad referencial e indices secundarios, se documenta en el Anexo C.

5.7. Contrato de la interfaz REST

La interfaz de programacion de aplicaciones expuesta por el modulo Python sigue el estilo arquitectonico REST (Fielding & Taylor, 2002), con representacion de recursos mediante identificadores uniformes, uso semantico de los metodos HTTP estandar y comunicacion sin estado del lado del servidor. La autentificacion se realiza mediante encabezados con tokens portadores firmados (Bearer tokens), validados en cada solicitud contra una clave compartida con el motor de orquestacion. La Tabla 5 sintetiza los principales puntos de entrada.

Tabla 5. Puntos de entrada principales de la interfaz REST

Metodo	Ruta	Codigo	Funcion
POST	<code>/api/v1/incidentes</code>	201 Created	Crea un nuevo ticket y devuelve el identificador generado.
GET	<code>/api/v1/incidentes/{id}</code>	200 OK	Recupera la informacion de un ticket por su identificador.
POST	<code>/api/v1/clasificar</code>	200 OK	Recibe descripcion textual y devuelve categoria sugerida con su confianza.
GET	<code>/api/v1/health</code>	200 OK	Verificacion de salud del servicio para monitoreo externo.

Los codigos de estado HTTP se utilizan conforme a su semantica original: 201 cuando un recurso se crea exitosamente, 200 cuando una consulta se atiende con exito, 400 cuando los datos de entrada no validan el esquema esperado, 401 cuando la autentificacion falla, y 500 cuando ocurre un error interno no recuperable. La especificacion completa en formato OpenAPI 3.1 se genera automaticamente por FastAPI y se encuentra disponible en el Anexo D.

6. Implementacion

6.1. Entorno de despliegue y dependencias

La implementacion del sistema se llevo a cabo utilizando exclusivamente tecnologias de codigo abierto con el objetivo de garantizar la portabilidad, la reproducibilidad y la auditoria de seguridad de la solucion en organizaciones medianas. Toda la infraestructura se encapsula mediante contenedores Docker, orquestados localmente por Docker Compose para los entornos de desarrollo y preproduccion, y preparados para migracion a un cluster Kubernetes 1.30 para el entorno productivo.

Las versiones especificas de los componentes son: N8N 1.62 sobre imagen oficial, Python 3.12 sobre imagen `python:3.12-slim`, PostgreSQL 15.5 sobre imagen `postgres:15.5-alpine` con volumen persistente, FastAPI 0.115, Uvicorn 0.32, SQLAlchemy 2.0, y el cliente `google-generativeai` 0.8 para la comunicacion con Gemini 2.5 Flash. La gestion de dependencias del modulo Python se realiza mediante un archivo `requirements.txt` con versiones fijadas (*pinned*) para garantizar la reproducibilidad de las compilaciones en distintos entornos.

6.2. Construcccion del modulo Python

La seleccion del marco de trabajo para el servicio de procesamiento implico la evaluacion de tres alternativas: Flask, Django REST Framework y FastAPI. Flask, ampliamente adoptado en el ecosistema Python por su simplicidad, carece de validacion nativa de esquemas y de generacion automatica de documentacion OpenAPI, funcionalidades que el proyecto requeria para garantizar la calidad de la interfaz REST. Django REST Framework, por su parte, ofrece un conjunto completo de herramientas para la construccion de interfaces de programacion de aplicaciones, pero su modelo de operacion sincronico y la inclusion de componentes no requeridos por el proyecto —tales como el ORM de Django y el motor de plantillas— incrementaban la complejidad de la base de codigo sin aportar beneficios funcionales para una arquitectura de microservicio acotado. FastAPI 0.115, sustentado sobre el servidor ASGI Uvicorn 0.32, ofrece tres ventajas determinantes para el proyecto: validacion automatica de esquemas de entrada y salida mediante Pydantic v2, generacion de documentacion interactiva conforme a la especificacion OpenAPI 3.1 sin configuracion adicional, y operacion asincronica nativa que permite manejar las llamadas concurrentes desde N8N sin bloquear el hilo de ejecucion principal (Tiangolo, 2024). La madurez del ecosistema de FastAPI, su adopcion creciente en la

industria y la disponibilidad de documentacion exhaustiva en espanol consolidaron la decision.

El modulo Python expone su contrato REST organizado en capas, segun la estructura descrita en el Capitulo 5, con una separacion clara entre responsabilidades:

- **models**: entidades del modelo de dominio mapeadas mediante SQLAlchemy 2.0.
- **schemas**: modelos de validacion y serializacion con Pydantic v2.
- **services**: logica de aplicacion (IncidenteService, ClasificacionService).
- **repositories**: acceso a datos, aislando las consultas SQL del resto de la aplicacion.
- **classifiers**: logica del clasificador hibrido (filtro deterministico + LLM).
- **routes**: enrutadores REST que delegan en los servicios correspondientes.
- **core**: configuracion de base de datos, manejo de errores y logging estructurado.

La construccion se realiza bajo un esquema de inyeccion de dependencias gestionado por las funcionalidades nativas de FastAPI (*Depends*), lo que facilita el reemplazo de implementaciones durante la ejecucion de pruebas unitarias y de integracion. Esta arquitectura permite, por ejemplo, sustituir el clasificador real por una implementacion simulada durante las pruebas sin modificar el codigo de los servicios que lo consumen. La gestion de credenciales se realiza exclusivamente mediante variables de entorno, en linea con el principio de configuracion de la metodologia Twelve-Factor App (Wiggins, 2017).

6.3. Construccion del flujo en N8N

El flujo de orquestacion implementado en N8N comprende 12 nodos principales encadenados secuencialmente. La eleccion de N8N como plataforma de orquestacion, frente a alternativas como Apache Airflow, Zapier o Make, obedecio a tres criterios concurrentes. El primero, de naturaleza operativa, radica en que N8N admite despliegue completamente autoalojado bajo Docker, manteniendo los flujos de procesamiento y los datos dentro del perimetro organizacional. Zapier y Make operan exclusivamente como servicios en la nube, lo que las descarta en escenarios donde el procesamiento de datos sensibles exige control de infraestructura. Apache Airflow, aunque autoalojable, disena sus flujos como grafos aciclicos dirigidos programados por intervalos temporales; ese paradigma resulta adecuado para procesos de extraccion, transformacion y carga de datos pero forzosamente inadecuado para un sistema reactivo que debe responder en tiempo real a eventos de entrada no programados (Apache Software Foundation, 2024). El segundo criterio refiere al modelo de construccion visual que N8N ofrece sobre un editor de nodos arrastrables: la curva de aprendizaje de esta interfaz es

sustancialmente menor que la de alternativas centradas en definicion programatica de flujos, lo que permitio al equipo concentrar el esfuerzo en la logica de negocio antes que en la sintaxis de definicion de procesos. El tercer criterio se vincula con la licencia de codigo fuente disponible (*fair-code*) que habilita la auditoria de seguridad del nucleo de orquestacion (n8n GmbH, 2024).

La composicion de los 12 nodos responde a un patron de procesamiento en cinco etapas que se replica para cada canal:

1. **Recepcion:** Tres disparadores (*triggers*) reciben las entradas desde correo electronico (IMAP), formulario web (Webhook) y telefonia (Webhook post-transcripcion). La decision de separar los disparadores por canal, en lugar de unificar la recepcion mediante un unico webhook con discriminacion interna, se fundamenta en la necesidad de preservar la configuracion de autenticacion y los parametros de reintento especificos de cada protocolo.
2. **Normalizacion:** Un nodo de transformacion (*Function*) homogeniza la estructura del mensaje en un formato unificado con campos de identificador unico (UUID v4), marca temporal, canal de origen, remitente y descripcion textual normalizada. Esta etapa elimina las diferencias de formato entre canales y garantiza que las capas posteriores procesen siempre la misma estructura de datos, independientemente de la via de entrada.
3. **Clasificacion:** Un nodo HTTP invoca el endpoint `POST /api/v1/clasificar` del modulo Python enviando la descripcion textual y recibiendo la categoria sugerida y la confianza asociada en formato JSON.
4. **Evaluacion:** Un nodo condicional (*IF*) evalua la confianza devuelta por el clasificador; si esta por debajo del umbral de 0,70, deriva el incidente a la cola de revision humana y notifica al operador designado. Si la confianza supera el umbral, el flujo continua automaticamente.
5. **Persistencia y notificacion:** Un nodo HTTP invoca `POST /api/v1/incidentes` para persistir el ticket en la base de datos, y tres nodos paralelos notifican al usuario por el canal correspondiente y registran la ejecucion en el sistema de auditoria. El paralelismo de la notificacion garantiza que la demora de un canal (por ejemplo, el envio de correo electronico) no bloquee el registro de auditoria ni la confirmacion por otros medios.

La trazabilidad completa de cada ejecucion se preserva almacenando los identificadores de nodo, las marcas temporales de entrada y salida de cada etapa, y los datos de entrada y salida de cada paso del flujo, generando un historial auditable que resulta especialmente valioso en contextos donde la Ley 25.326 exige demostrar el camino recorrido por los datos personales (Honorable Congreso de la Nación Argentina, 2000). La exportacion completa del flujo en formato JSON se incluye en el Anexo E.

6.4. Integracion del canal telefonico

La integracion con Twilio se realizo mediante el producto Programmable Voice, complementado con la transcripcion automatica nativa. El flujo telefonico opera de la siguiente manera: el usuario marca un numero virtual asignado a la organizacion; un script TwiML reproduce un mensaje de bienvenida en espanol rioplatense y solicita al usuario describir su problema durante un maximo de 45 segundos; la grabacion finaliza cuando el usuario presiona la tecla numeral (#) o transcurrido el tiempo maximo; la grabacion se envia al servicio de transcripcion de Twilio; y, una vez transcrito el contenido, Twilio invoca un webhook expuesto por N8N con la transcripcion textual. A partir de ese punto, el flujo continua de manera identica a la de los canales escritos.

La latencia total del canal telefonico —medida desde el cuelgue del usuario hasta la confirmacion del numero de incidente— se ubico en un rango medio de 12 a 15 segundos segun las observaciones operativas durante la fase de validacion. Esta latencia incluye: transcripcion (5–7 s), clasificacion (2–3 s para incidentes resueltos por el filtro deterministico, 5–8 s cuando se consulta al LLM) y persistencia (1 s).

6.5. Desafios de integracion

La implementacion del sistema enfrento tres desafios de integracion que merecen documentacion explicita por su potencial utilidad en replicas futuras.

El primer desafio se presento en la comunicacion asincronica entre N8N y el servicio de clasificacion. El nodo HTTP de N8N invoca el endpoint de clasificacion de manera sincronica y espera la respuesta del servidor antes de continuar el flujo. Esta arquitectura, aunque simple de implementar, introduce un punto de fallo unico: si el servicio Python no responde dentro del tiempo de espera configurado en el nodo HTTP, el flujo completo se detiene y el incidente queda sin procesar. La solucion consistio en implementar un mecanismo de reintento con

retroceso exponencial en el propio nodo HTTP de N8N, combinado con una cola de respaldo en el servicio Python que retiene los mensajes recibidos durante ventanas de saturación y los procesa en orden cuando la carga se normaliza. Esta combinación de reintento en el cliente y cola en el servidor eliminó las pérdidas de incidentes observadas durante las pruebas de carga iniciales.

El segundo desafío surgió de la heterogeneidad de los formatos de entrada. Los correos electrónicos incluyen encabezados, firmas corporativas y reenvíos que contaminan el texto que el clasificador recibe; las llamadas telefónicas transcritas introducen errores de reconocimiento del habla, particularmente en términos técnicos (por ejemplo, `API` transcrito como `a p i o a p e y e`); y los formularios web, aunque proveen texto limpio, llegan con una proporción significativa de campos vacíos o con descripciones de una sola palabra. La respuesta a esta heterogeneidad fue doble. Por un lado, se implementó un preprocesador en el nodo de normalización de N8N que elimina firmas, encabezados de reenvío y líneas de disclaimer mediante expresiones regulares específicas del dominio de correo institucional. Por el otro, se enriqueció el prompt enviado al clasificador con instrucciones explícitas para interpretar términos técnicos frecuentemente mal transcritos, lo que elevó la robustez de la clasificación sobre transcripciones automáticas en aproximadamente 4 puntos porcentuales de F1 respecto de la configuración inicial sin estas instrucciones.

El tercer desafío, de naturaleza operativa, se vincula con la persistencia transaccional de la sesión de clasificación. El sistema debe garantizar que una vez que el clasificador asigna una categoría, el incidente se persiste con esa decisión o, en caso de falla de la base de datos, el estado interno del sistema refleje la inconsistencia y pueda recuperarse. La solución adoptó un enfoque pragmático: N8N ejecuta la clasificación y la persistencia como pasos secuenciales dentro del mismo flujo; si la persistencia falla, el flujo se reintenta desde el paso de clasificación, lo que garantiza que el incidente eventualmente se registre aun a costa de duplicar la llamada al clasificador. La tabla de trazabilidad (`clasificacion_log`) fue diseñada para admitir múltiples registros de clasificación por incidente, preservando así el historial de cada intento.

6.6. Conexión con el proceso Scrumban

La construcción del sistema fue organizada bajo el esquema Scrumban descrito en el Capítulo 4, ejecutando seis iteraciones de dos semanas cada una. La vinculación entre cada

iteracion y los artefactos de implementacion producidos se detalla a continuacion:

Iteracion 1: La elicitation de requisitos (Semanas 1–2) produjo el disenio preliminar de la arquitectura de cinco capas y la especificacion de la interfaz REST, documentadas en el Capitulo 5. El entregable concreto fue el documento de arquitectura y la definicion del contrato OpenAPI inicial.

Iteracion 2: La construccion del modulo Python (Semanas 3–4) entrego el modelo de datos con sus cinco entidades, los repositorios con operaciones CRUD asincronicas y los servicios de negocio. Al cierre de esta iteracion, el modulo Python exponia el endpoint de salud y aceptaba la creacion de incidentes mediante `POST /api/v1/incidentes`, con cobertura de pruebas unitarias superior al 80 %.

Iteracion 3: La configuracion inicial del flujo N8N (Semanas 5–6) implemento el canal de correo electronico como via prioritaria, estableciendo el patron de cinco etapas que luego se replicaria para los canales restantes. Al finalizar esta iteracion, el sistema procesaba incidentes por correo electronico de extremo a extremo, desde la recepcion hasta la notificacion.

Iteracion 4: La integracion del canal telefonico y la persistencia (Semanas 7–8) incorporo Twilio Programmable Voice, el script TwiML y la configuracion del webhook de transcripcion. Esta iteracion fue la que mas expuso los desafios de heterogeneidad de formatos descritos en la Seccion 6.5, y su resolucion consumio aproximadamente el 40 % del esfuerzo de la iteracion.

Iteracion 5: La validacion funcional y el ajuste fino del clasificador (Semanas 9–10) se concentraron en la calibracion del filtro deterministico y del prompt de Gemini. Los ajustes se realizaron exclusivamente sobre incidentes representativos en entornos de preproduccion, sin exposicion al corpus de validacion de 200 casos, preservando la integridad de la etapa experimental posterior.

Iteracion 6: La validacion experimental y la documentacion final (Semanas 11–12) ejecutaron el protocolo de pruebas descrito en el Capitulo 4, recolectaron las mediciones del corpus completo y produjeron el analisis estadistico presentado en el Capitulo 7.

La trazabilidad entre iteraciones y artefactos, mediada por el tablero Kanban en GitHub

Projects, permitio al equipo identificar tempranamente los desvios respecto del plan y redistribuir el esfuerzo cuando la complejidad de un componente superaba la estimacion inicial. La adopcion de limites de trabajo en curso (WIP) por columna del tablero evito la acumulacion de tareas a medio terminar, una practica que resulto particularmente valiosa durante las iteraciones 4 y 5, donde la interdependencia entre modulos exigia finalizar componentes antes de integrarlos.

6.7. Pruebas automatizadas

La estrategia de pruebas se sustento en la piramide clasica descrita por Crispin y Gregory (2009). En la base se ubicaron las pruebas unitarias del modulo Python construidas con el marco pytest 8.3, alcanzando una cobertura del 87 % del codigo de aplicacion medida con coverage.py 7.6. Estas pruebas cubren los clasificadores (deterministico e hibrido), los servicios de negocio, los esquemas de validacion y los repositorios de acceso a datos.

En el nivel intermedio se ejecutaron pruebas de integracion que verificaron la interaccion entre el modulo Python y la base de datos en una instancia de PostgreSQL desechable creada por contenedor en cada corrida de pruebas, utilizando SQLite en memoria para las pruebas unitarias y PostgreSQL real para las pruebas de integracion. En el nivel superior se construyeron pruebas extremo a extremo que ejecutaban el flujo completo desde el envio de un correo simulado hasta la persistencia del ticket, validando la integracion con N8N en su totalidad.

La integracion continua (CI) se ejecuta automaticamente en cada solicitud de incorporacion (*pull request*) al repositorio mediante GitHub Actions, ejecutando la suite completa de pruebas unitarias y de integracion antes de permitir la fusion a la rama principal. El pipeline de CI incluye verificacion de estilo de codigo (Ruff), ejecucion de pruebas con reporte de cobertura, y verificacion de sincronizacion de la especificacion OpenAPI.

7. Resultados

Este capítulo presenta los resultados de la validación experimental del sistema. Se reportan los hallazgos en cuatro dimensiones: tiempos de registro, desempeño de clasificación, reducción de la intervención humana y análisis cualitativo de errores.

7.1. Comparación de tiempos de registro

La comparación de los tiempos de registro entre el flujo manual y el flujo automatizado se sintetiza en la Tabla 6. La media aritmética del tiempo manual sobre los 200 casos del corpus fue de 165,3 s (equivalente a 2 min 45 s), con un desvío estándar de 38,7 s y un rango entre 96 y 289 s. La media del flujo automatizado fue de 18,2 s con un desvío estándar de 4,1 s y un rango entre 11 y 31 s. La reducción relativa de la media de tiempo alcanzó el 89,0 %.

Tabla 6. Estadísticos descriptivos de los tiempos de registro por flujo ($n = 200$)

Estadístico	Flujo manual	Flujo automatizado	Reducción
Media aritmética (s)	165,3	18,2	89,0 %
Mediana (s)	158,0	17,4	89,0 %
Desvío estándar (s)	38,7	4,1	—
Mínimo (s)	96	11	—
Máximo (s)	289	31	—
IC 95 % de la media (s)	[159,9; 170,7]	[17,6; 18,8]	—

La prueba de Wilcoxon de rangos con signo aplicada sobre las mediciones pareadas arrojó un estadístico $W = 0$ con 200 pares válidos y un valor $p < 0,001$, lo que permite rechazar la hipótesis nula de igualdad de medianas con un nivel de confianza superior al 99,9 %. El estadístico $W = 0$ implica que en los 200 pares el flujo automatizado fue siempre más veloz que el manual, sin una sola excepción, resultado plausible dada la diferencia absoluta de aproximadamente 147 s entre medianas.

Para cuantificar la magnitud práctica de la diferencia, se calculó el coeficiente r de correlación rank-biserial:

$$r = 1 - \frac{2W}{n(n+1)} = 1 - \frac{0}{200 \cdot 201} = 1,00$$

El valor obtenido ($r = 1,00$) corresponde al tamaño del efecto máximo posible, consistente con la ausencia de pares invertidos. La diferencia observada es, por tanto, estadísticamente

significativa y, dada su magnitud absoluta, relativa y el tamaño del efecto reportado, también prácticamente relevante para la operación.

7.2. Matriz de confusión y métricas de clasificación

La matriz de confusión obtenida sobre los 200 casos del corpus se presenta en la Tabla 7. De los 82 casos de Sistemas, 76 fueron correctamente clasificados (92,7 %), 4 se derivaron a Operaciones y 2 a Soporte Técnico. De los 64 casos de Operaciones, 58 fueron correctos (90,6 %), 3 se derivaron a Sistemas y 3 a Soporte Técnico. De los 54 casos de Soporte Técnico, 50 fueron correctos (92,6 %), 2 se derivaron a Sistemas y 2 a Operaciones. La exactitud global resultante asciende al 92,0 % (184 casos correctos sobre 200).

Tabla 7. Matriz de confusión del clasificador automático ($n = 200$)

Real \ Predicho	Sistemas	Operaciones	Soporte Técnico	Total
Sistemas	76	4	2	82
Operaciones	3	58	3	64
Soporte Técnico	2	2	50	54
Total	81	64	55	200

Los valores de precisión, sensibilidad (*recall*) y F1 calculados por clase, junto con su promedio macro, se presentan en la Tabla 8. La clase Sistemas obtuvo el valor F1 más alto (0,933), seguida por Soporte Técnico (0,917) y Operaciones (0,906). El promedio macro de F1 alcanza 0,919, valor sustancialmente superior al umbral de 0,85 planteado en el segundo objetivo específico del trabajo. La exactitud global del 92 % (184/200) presenta un intervalo de confianza al 95 %, estimado por el método de Wilson, de [87,2 %, 95,2 %], cuyo límite inferior supera ampliamente el umbral objetivo del 85 %.

Tabla 8. Métricas de clasificación por clase y promedio macro

Clase	Precisión	Sensibilidad	F1
Sistemas	0,938	0,927	0,933
Operaciones	0,906	0,906	0,906
Soporte Técnico	0,909	0,926	0,917
Macro promedio	0,918	0,920	0,919

Estas métricas confirman que el clasificador exhibe un desempeño equilibrado entre las

tres clases objetivo, sin sesgos significativos hacia ninguna de ellas en particular. La dispersion de los valores F1 entre clases —con un rango de solo 0,027 puntos— sugiere que el sistema no favorece sistemáticamente a la clase mayoritaria (Sistemas, con 82 casos) en detrimento de las minoritarias. Este equilibrio es particularmente relevante en el contexto operativo: un clasificador que maximiza la exactitud global a costa de un desempeño pobre en una clase minoritaria genera derivaciones erróneas concentradas en un sector específico, lo que erosiona la confianza de ese equipo en el sistema y puede llevar al rechazo operativo de la herramienta, independientemente de sus métricas agregadas.

La inspección de la matriz de confusión revela un dato que las métricas agregadas no capturan completamente: de los 16 errores totales, exactamente la mitad (8) corresponden a confusiones entre Sistemas y Operaciones. Este patrón no es aleatorio. Ambas categorías comparten un espacio léxico considerable —infraestructura, conectividad, permisos de acceso— y los incidentes que las involucran frecuentemente requieren un diagnóstico inicial que el texto de la descripción no proporciona. Por ejemplo, un incidente reportado como “no puedo entrar al sistema” puede originarse en un error de autenticación (Operaciones), en una caída del servidor de aplicaciones (Sistemas) o en una falla de conectividad de red (Operaciones). En ausencia de información contextual que un usuario no siempre incluye en la descripción inicial, el clasificador enfrenta una ambigüedad estructural que no se resuelve con mejoras incrementales del modelo sino con un enriquecimiento del protocolo de captura del incidente. Esta observación orienta directamente una de las recomendaciones de trabajo futuro: incorporar preguntas de refinamiento tempranas que, sin convertir el proceso en un árbol de decisión exhaustivo, permitan al clasificador reducir la zona de ambigüedad entre las dos categorías más propensas a confusión.

7.3. Reducción de la intervención humana

La medición de la intervención humana se realizó cronometrando, sobre cada caso, los segundos en que el operador ejecutó acciones manuales sobre el sistema. En el flujo manual, el 100 % del tiempo correspondió a intervención humana, incluyendo lectura, comprensión, búsqueda en sistemas auxiliares y carga del ticket. En el flujo automatizado, el operador humano solo intervino en aquellos casos en los que el clasificador devolvió una confianza inferior al 70 % o en los que el sector responsable, al recibir el ticket, identificó una clasificación incorrecta y la corrigió antes de iniciar la atención.

La proporcion agregada de intervencion humana en el flujo automatizado se ubico en el 9,5 %. Este valor es coherente con la propuesta de un esquema *human in the loop* donde el sistema asume la mayoria de los casos rutinarios y reserva al operador la atencion de los casos complejos o ambiguos. De los 200 casos procesados, 181 (90,5 %) fueron clasificados y registrados sin intervencion humana, mientras que 19 (9,5 %) requirieron verificacion o correccion por parte de un operador.

7.4. Analisis de errores y casos limite

El analisis cualitativo de los 16 casos mal clasificados permitio identificar tres patrones de error recurrentes, sintetizados en la Tabla 9.

Tabla 9. Tipologia de errores observados en la clasificacion automatica

Patron	Casos	Descripcion
1. Descripciones cortas	7	Incidentes con menos de 15 palabras donde la falta de contexto lexico llevo al clasificador a decisiones equivocadas.
2. Mezcla de categorias	6	Incidentes donde una falla de hardware afecta a una aplicacion especifica; la decision correcta depende del juicio humano sobre cual aspecto prevalece.
3. Jerga local	3	Incidentes con uso intensivo de jerga organizacional o nombres internos de sistemas no presentes en el contexto del prompt enviado al modelo.

El primer patron, presente en 7 casos (43,8 % de los errores), corresponde a incidentes con descripciones extremadamente cortas. La ausencia de contexto lexico suficiente impide tanto al filtro deterministico como al LLM identificar la categoria correcta con confianza. El segundo patron, presente en 6 casos (37,5 %), corresponde a incidentes que contienen elementos lexicos de dos categorias simultaneamente. El tercer patron, presente en 3 casos (18,8 %), refleja una limitacion inherente del enfoque basado en modelo externo: la ausencia de conocimiento organizacional especifico que un operador humano posee por experiencia.

Esta tipologia orienta directamente las recomendaciones de trabajo futuro presentadas en el Capitulo 10, particularmente la incorporacion de un clasificador supervisado entrenado con el corpus etiquetado generado por el propio sistema y la inclusion de un glosario de terminos organizacionales en el prompt del modelo.

8. Discussion

8.1. Interpretacion de los resultados

Los resultados obtenidos confirman la hipotesis de trabajo formulada en la Seccion 1.4. La reduccion del 89 % en el tiempo medio de registro y la exactitud global del 92 % alcanzada por el clasificador automatico superan ampliamente los umbrales planteados en los objetivos especificos del trabajo. Tres aspectos de los resultados merecen una interpretacion detallada.

La reduccion de tiempo se sostiene incluso al incorporar el costo de inferencia del modelo de lenguaje grande externo, dado que la primera etapa deterministica del clasificador resuelve aproximadamente el 62 % de los casos sin invocar al modelo de Google. Este hallazgo valida la hipotesis subsidiaria sobre la eficiencia del esquema hibrido: el filtro deterministico no solo reduce el costo economico del sistema, sino que ademas disminuye la latencia mediana del flujo automatizado al evitar llamadas de red innecesarias al servicio externo. Desde una perspectiva operativa, esta propiedad del clasificador introduce un punto de inflexion en la relacion costo-beneficio de los sistemas basados en LLM: el costo marginal de la inferencia se aplica unicamente a los casos que realmente demandan capacidad semantica, y no al volumen completo de incidentes.

La dispersion observada en los tiempos del flujo automatizado —con un rango de 11 a 31 segundos, frente a los 96 a 289 segundos del flujo manual— evidencia una compresion sustancial de la variabilidad. Mientras que en el flujo manual un incidente podia demandar entre 96 y 289 segundos segun la complejidad percibida y la carga cognitiva del operador, en el flujo automatizado el rango se comprime a 11–31 segundos, con una varianza casi diez veces menor. Esta compresion de la variabilidad no fue un objetivo explicito del diseno pero emerge como una consecuencia deseable de la automatizacion: el sistema no solo es mas rapido en promedio, sino tambien mas consistente.

La reduccion de la intervencion humana al 9,5 %, lejos de ser una limitacion, materializa el esquema *human in the loop* buscado: el sistema absorbe la carga rutinaria sin sustituir el juicio humano en los casos que verdaderamente lo demandan. Este equilibrio entre automatizacion y supervision preserva la calidad del servicio y libera capacidad operativa significativa. Si se proyecta este porcentaje sobre el volumen trimestral de 3.700 incidentes, la automatizacion liberaria aproximadamente 180 horas-persona por trimestre, tiempo que el personal podria redirigir a tareas de resolucion tecnica o a actividades de mejora continua. La magnitud de esta

liberacion —equivalente a un mes completo de jornada laboral de un operador— transforma la automatizacion del registro de un proyecto de optimizacion marginal en una intervencion con impacto organizacional medible.

El rigor metodologico adoptado para evitar *data leakage* merece una observacion aparte. A diferencia de trabajos que particionan el corpus en conjuntos de entrenamiento y prueba despues de realizar exploraciones iniciales —practica que puede introducir fuga de informacion—, el presente estudio aislo completamente la construccion de los componentes criticos del clasificador del corpus de validacion de 200 casos. El filtro deterministico se construyo mediante analisis manual de incidentes no incluidos en el corpus, y el prompt fue ajustado en entornos de preproduccion sin exposicion a los 200 casos de validacion. Solo despues de completar el desarrollo se evaluo el desempeno sobre la totalidad del corpus, garantizando que las metricas reportadas reflejan el desempeno genuino del sistema sobre datos no vistos. Esta separacion estricta entre datos de construccion y datos de validacion constituye una garantia metodologica que no todos los trabajos del area documentan con el mismo nivel de explicitacion.

8.2. Comparacion con la literatura

Los valores de exactitud y F1 macro obtenidos resultan consistentes con los reportados en la literatura comparable. Paramesh y Shreedhara (2019) reportan exactitudes cercanas al 87 % utilizando SVM sobre representaciones TF-IDF, mientras que Revina et al. (2020) alcanzan el 91 % con arquitecturas basadas en BERT. Evaluaciones recientes de LLM en escenarios *few-shot* para clasificacion de tickets reportan valores entre el 88 % y el 94 % de exactitud dependiendo del idioma y del dominio.

Los resultados del presente trabajo —92 % de exactitud global y F1 macro de 0,919— se ubican dentro del rango superior reportado para idiomas relativamente bien representados en los corpus de entrenamiento de los modelos contemporaneos. Dos comparaciones resultan particularmente informativas. Frente al enfoque SVM+TF-IDF de Paramesh y Shreedhara (2019) (87 %), el clasificador hibrido propuesto ofrece una ganancia de 5 puntos porcentuales en exactitud, ganancia que resulta operativamente relevante si se considera que cada error de clasificacion implica entre 10 y 15 minutos adicionales de reasignacion. Frente al enfoque BERT de Revina et al. (2020) (91 %), la diferencia es de solo 1 punto porcentual, pero el esquema propuesto no requiere infraestructura de GPU para inferencia local ni un corpus de entrenamiento etiquetado de gran escala, lo que lo hace mas accesible para organizaciones

medianas.

La novedad del aporte radica en la validación específica sobre español rioplatense aplicado a un dominio de mesa de ayuda de organización mediana, segmento escasamente cubierto por la literatura previa. Diversos trabajos han señalado la menor cobertura del español en los corpus de entrenamiento de los LLM contemporáneos, y los resultados del presente trabajo ($F1 \text{ macro} = 0,919$) sugieren que, al menos para el subdominio de tickets de soporte, el desempeño en español rioplatense es competitivo con el reportado para idiomas de mayor cobertura como el inglés o el alemán.

8.3. Limitaciones del estudio

El estudio presenta cinco limitaciones que deben considerarse al interpretar los hallazgos y al planificar replicaciones futuras.

Primero, el tamaño de la muestra ($n = 200$) es suficiente para una validación piloto pero insuficiente para sustentar generalizaciones de carácter poblacional sobre el universo de mesas de ayuda en organizaciones medianas. Si bien el intervalo de confianza del 95 % reportado para la exactitud [87,2 %, 95,2 %] no solapa la zona de rechazo, muestras mayores permitirían intervalos más estrechos y mayor potencia para detectar diferencias finas entre clases.

Segundo, la circunscripción del corpus a una única organización del sector servicios de la provincia de Mendoza restringe la validez externa de los hallazgos. La distribución de tipos de incidentes, el léxico organizacional y los patrones de redacción pueden variar significativamente entre organizaciones, sectores y regiones.

Tercero, la dependencia operativa de un servicio externo de inferencia condiciona el rendimiento del sistema. Un eventual deterioro del servicio de Gemini 2.5 Flash, un cambio significativo en las condiciones comerciales del proveedor, o una modificación en el comportamiento del modelo debido a actualizaciones afectaría directamente la operación. Esta dependencia es intrínseca al enfoque basado en LLM externo y debe ser gestionada como un riesgo operativo.

Cuarto, los resultados reflejan el comportamiento del modelo en su versión vigente al momento del estudio (junio de 2026). Las actualizaciones futuras del proveedor pueden modificar tanto el desempeño bruto como la estructura del prompt requerido para alcanzar resultados equivalentes, lo que aconseja la implementación de un régimen de monitoreo continuo de las

metricas de clasificacion.

Quinto, la comparacion se realizo exclusivamente contra el flujo manual (proceso operativo), no contra clasificadores alternativos como TF-IDF+SVM o regresion logistica sobre el mismo corpus. Una evaluacion preliminar de un clasificador TF-IDF+SVM sobre un subconjunto de evaluacion arrojó una exactitud del 78 % y un F1 macro de 0,764, lo que confirma que el esquema hibrido propuesto aporta una ganancia sustantiva sobre un *baseline* simple. Investigaciones futuras deberian incluir una comparacion sistematica contra un abanico mas amplio de clasificadores sobre el corpus completo.

8.4. Implicancias practicas

Desde el punto de vista practico, los resultados sugieren que organizaciones medianas con volumenes de incidentes comparables al observado en la organizacion analizada pueden obtener beneficios sustanciales mediante la implementacion de soluciones hibridas similares a la propuesta. La reduccion de la intervencion humana en la etapa de registro libera al personal tecnico para concentrarse en la resolucion efectiva de los casos, etapa en la cual el juicio experto continua siendo insustituible.

La trazabilidad provista por el registro detallado de cada decision del clasificador habilita dos procesos de mejora continua. Por un lado, permite identificar patrones de error sistematicos y ajustar el filtro deterministico o el prompt del LLM en consecuencia. Por otro lado, la acumulacion sostenida de decisiones validadas por los sectores responsables constituye un activo informacional que, una vez alcanzado un volumen adecuado (del orden de 5.000 casos), permitiria entrenar modelos de clasificacion especificos del dominio organizacional, con eventuales ventajas en costo, latencia y soberania sobre los datos.

En el plano economico, el costo operativo del sistema se compone de dos rubros principales: el costo de infraestructura autoalojada (servidor con Docker, estimado en 50–100 USD/mes segun el proveedor de nube local) y el costo de inferencia del LLM (aproximadamente 0,02 USD por cada invocacion a Gemini 2.5 Flash en la banda de uso correspondiente). Para un volumen de 3.700 incidentes trimestrales, con un 38 % requiriendo inferencia externa, el costo trimestral de API se estima en el orden de 28 USD, un valor marginal frente a las 180 horas-persona liberadas.

8.5. Reflexiones sobre la generalizacion

La generalizacion de los resultados a otras organizaciones requiere atender al menos tres condiciones. En primer lugar, la disponibilidad de un corpus etiquetado de tamaño y calidad similares al utilizado en este trabajo, lo cual implica una inversion inicial en doble etiquetado y validacion del acuerdo inter-evaluador. En segundo lugar, la viabilidad tecnica de mantener desplegada localmente la infraestructura de orquestacion y persistencia, condicion que descarta a organizaciones sin capacidades operativas minimas para la administracion de contenedores Docker. En tercer lugar, la existencia de un marco normativo compatible con el envio de fragmentos descriptivos de incidentes a un servicio externo de inferencia, marco que en el caso argentino se encuentra contemplado por la Ley 25.326 (Honorable Congreso de la Nación Argentina, 2000) bajo determinadas condiciones de minimizacion y pseudonimizacion descritas en el Capitulo 11.

9. Conclusiones

El trabajo desarrollado ha cumplido satisfactoriamente con el objetivo general planteado en la Sección 1.5: construir un sistema automatizado de mesa de ayuda capaz de recibir, procesar, clasificar y registrar incidentes de manera automática, reduciendo la intervención humana en la etapa inicial. La integración de N8N como motor de orquestación, un módulo Python con FastAPI como núcleo de procesamiento, PostgreSQL como capa de persistencia, Twilio como canal telefónico y el modelo Gemini 2.5 Flash como motor de inferencia lingüística ha demostrado ser una combinación arquitectónica viable, eficiente y económicamente accesible para organizaciones medianas.

9.1. Cumplimiento de los objetivos del trabajo

El diseño arquitectónico del sistema materializó los principios de separación de responsabilidades y cohesión interna que la disciplina de ingeniería de software prescribe para sistemas distribuidos. La arquitectura de cinco capas —canales de entrada, orquestación, procesamiento, inferencia y persistencia— distribuyó las responsabilidades entre componentes independientes comunicados mediante interfaces REST sobre TLS 1.3. La elección de tecnologías maduras y de despliegue autoalojado (Docker, PostgreSQL, N8N) no fue meramente una decisión instrumental sino una respuesta directa a las restricciones del contexto: organizaciones medianas argentinas que operan con presupuestos acotados, enfrentan limitaciones cambiarias para la adquisición de software comercial y están sujetas a un marco normativo que exige control sobre los datos personales.

Sobre esa arquitectura se construyó el clasificador híbrido, cuyo desempeño superó con holgura los umbrales establecidos en el segundo objetivo específico. La exactitud global del 92 % y el F1 macro de 0,919 superan en siete y siete puntos porcentuales respectivamente las cotas mínimas fijadas. Pero más allá de la magnitud de las métricas, lo que los resultados revelan es la eficacia de la estrategia compositiva adoptada: combinar un filtro determinístico de primera etapa con un modelo de lenguaje grande invocado únicamente cuando el filtro no alcanza el umbral de confianza. Esta arquitectura de dos etapas produjo un sistema donde aproximadamente seis de cada diez incidentes se resuelven sin costo de inferencia externa, sin latencia de red y sin transferencia de datos a infraestructura de terceros. Los tres restantes, aquellos que presentan ambigüedad semántica o redacción atípica, se benefician de la capacidad de comprensión contextual del modelo de lenguaje. La distribución del trabajo entre

las dos etapas no fue un resultado buscado sino una propiedad emergente del diseño, y sin embargo resultó ser uno de los hallazgos más valiosos del trabajo: demuestra que los LLM, correctamente enmarcados como árbitros de último recurso y no como clasificadores universales, ofrecen un punto de equilibrio notable entre costo, latencia y exactitud.

La integración de los tres canales paralelos dentro de un único flujo de orquestación confirmó que la diversidad de vías de entrada no introduce dispersión operativa cuando la normalización se realiza tempranamente en el flujo. El canal telefónico, que a priori representaba el mayor desafío técnico por la inclusión de transcripción automática del habla, alcanzó una latencia de extremo a extremo de 12 a 15 segundos —un valor que lo posiciona como una vía prácticamente útil y no como una demostración de laboratorio. La decisión de normalizar la entrada en el propio N8N, antes de invocar al servicio de clasificación, resultó acertada: aisló la complejidad de cada protocolo de recepción del núcleo de procesamiento, permitiendo que el servicio Python operara siempre sobre la misma estructura de datos independientemente del canal de origen.

La evaluación comparativa entre el flujo manual y el automatizado arrojó resultados que trascienden lo meramente estadístico. La reducción de 165,3 s a 18,2 s en el tiempo medio de registro ($p < 0,001$, $r = 1,00$) no deja espacio para la duda razonable: en las condiciones del experimento, el sistema automatizado fue invariablemente más rápido que el operador humano, en los 200 pares medidos. La prueba de Wilcoxon con $W = 0$ es, en sí misma, un resultado poco frecuente en la investigación aplicada y merece ser interpretada con cautela: no implica que el sistema sea perfecto, sino que la diferencia con el proceso manual es tan abrumadora que ningún par invirtió el orden. La reducción de la intervención humana al 9,5 % completa el cuadro: el sistema no solo es más veloz sino que libera al personal técnico de la carga rutinaria de registro, redirigiendo ese tiempo hacia la resolución efectiva de los casos.

La documentación generada —arquitectura, procedimientos de despliegue, especificación OpenAPI, flujo N8N exportable y análisis tipificado de errores— constituye un activo que excede el cumplimiento del quinto objetivo específico. La inclusión de una tipología de errores con tres patrones identificados (descripciones cortas, mezcla de categorías y jerga local) proporciona un punto de partida concreto para cualquier organización que desee replicar o adaptar el sistema, permitiéndole anticipar las categorías de fallo más probables antes de iniciar su propia implementación.

9.2. Aportes y generalizacion

Mas alla del cumplimiento de los objetivos especificos, este trabajo permite formular una afirmacion general de conocimiento relevante para el campo de la automatizacion de mesas de ayuda: los modelos de lenguaje grandes invocados mediante instrucciones contextualizadas —sin ajuste fino sobre datos del dominio— son capaces de alcanzar niveles de exactitud en clasificacion de tickets de soporte superiores a los de clasificadores supervisados clasicos entrenados sobre representaciones estaticas, aun en idiomas de menor cobertura como el espanol rioplatense, siempre que se combinen con un filtro deterministico de primer nivel que absorba los casos triviales y reduzca el costo de inferencia.

Esta combinacion arquitectonica —orquestacion visual, filtrado deterministico y LLM como arbitro semantico— constituye un patron de diseno reproducible para el dominio de gestion de servicios en organizaciones medianas de America Latina, donde la restriccion presupuestaria y la soberania de datos son determinantes para la adopcion tecnologica. La arquitectura propuesta no depende de infraestructura especializada de GPU ni de licencias de software propietario, y su costo operativo esta dominado por el consumo de API de inferencia, que para el volumen observado (3.700 incidentes trimestrales) representa un costo marginal de aproximadamente 28 USD, una fraccion minima de las 180 horas-persona liberadas por trimestre.

El trabajo aporta tres contribuciones que se sostienen mas alla del caso particular. La primera es una arquitectura reproducible documentada en todos sus niveles, desde el modelo de datos hasta el contrato REST y el flujo de orquestacion, que cualquier equipo con conocimientos basicos de Docker, Python y N8N puede replicar. La segunda es evidencia empirica, recolectada bajo un diseno experimental riguroso con doble etiquetado independiente y prueba no parametrica, sobre el desempeno de LLM en un dominio y una variante linguistica escasamente documentados en la literatura. La tercera es un marco de implementacion que contempla desde el inicio las restricciones normativas argentinas, ofreciendo un camino de adopcion que no requiere sortear barreras legales ni asumir riesgos de cumplimiento.

10. Recomendaciones y líneas de trabajo futuro

A partir de los resultados obtenidos, del análisis de errores presentado en la Sección 7.4 y de las limitaciones reconocidas en la Sección 8.3, se formulan las siguientes recomendaciones para la evolución del sistema y para futuras replicaciones del estudio.

10.1. Incorporación de un clasificador supervisado con corpus propio

Se sugiere incorporar progresivamente un clasificador supervisado entrenado con el corpus etiquetado generado por el propio sistema durante su operación. La acumulación sostenida de decisiones validadas por los sectores responsables constituye un activo informacional valioso que, una vez alcanzado un volumen del orden de 5.000 casos, permitiría entrenar modelos de clasificación específicos del dominio organizacional. Las ventajas potenciales de esta migración incluyen: reducción del costo marginal por incidente clasificado al eliminar la dependencia del servicio externo de inferencia, disminución de la latencia al ejecutar la inferencia localmente, y ganancia de soberanía sobre los datos al evitar la transferencia internacional de fragmentos textuales. Touvron et al. (2023) y trabajos subsiguientes sobre modelos de código abierto sugieren que los LLM desplegados localmente alcanzan rendimientos competitivos en tareas de clasificación cuando se invocan mediante prompts adecuadamente diseñados, lo que abre una vía concreta hacia un sistema completamente autoalojado.

10.2. Ampliación de canales de entrada

Se propone ampliar el conjunto de canales de entrada para incluir mensajería corporativa instantánea, particularmente Slack y Microsoft Teams en aquellas organizaciones donde estas plataformas constituyen el canal de comunicación dominante. La inclusión de aplicaciones móviles propias para usuarios de campo constituye una extensión natural que atiende la demanda creciente de canales asincrónicos integrados al ecosistema laboral cotidiano. La arquitectura modular propuesta en el Capítulo 5 facilita esta ampliación: cada nuevo canal requiere únicamente un disparador adicional en N8N, sin modificaciones en las capas de procesamiento ni de persistencia.

10.3. Panel de monitoreo en tiempo real

Se sugiere implementar un panel de monitoreo sustentado sobre tecnologías de observabilidad maduras, tales como Prometheus para la captura de métricas y Grafana para

la visualización. Este panel permitiría supervisar la latencia del sistema, la tasa de aciertos del clasificador, la distribución de carga entre sectores y el costo agregado de las invocaciones al modelo externo, habilitando ajustes operativos basados en evidencia continua y alertas tempranas frente a degradaciones del servicio. La existencia de endpoints de salud (`GET /api/v1/health`) en cada componente del sistema proporciona los puntos de datos necesarios para instrumentar esta capa de observabilidad.

10.4. Mecanismos de aprendizaje activo

Se propone incorporar mecanismos de aprendizaje activo (*active learning*), en virtud de los cuales aquellos casos clasificados con baja confianza se prioricen automáticamente para revisión humana, y los resultados de esa revisión retroalimenten al sistema mediante un mecanismo de ajuste incremental. Este enfoque maximiza el rendimiento marginal de cada intervención humana —que de todos modos ocurre en el 9,5 % de los casos— y acelera la mejora continua del clasificador con un costo operativo controlado.

10.5. Estudios de replicación

Se sugiere realizar estudios de replicación en organizaciones de distinto tamaño, sector y región geográfica, con el objeto de fortalecer la validez externa de los hallazgos y construir un cuerpo de evidencia empírica sobre el desempeño de soluciones similares en contextos heterogéneos. Particularmente relevantes serían las réplicas en organizaciones del sector público (con mayor proporción de incidentes vinculados a sistemas administrativos), en organizaciones del sector industrial (con mayor proporción de incidentes vinculados a infraestructura física) y en organizaciones de mayor tamaño donde el volumen diario de incidentes supere las 100 unidades.

10.6. Evaluación de modelos de lenguaje de código abierto

Se propone evaluar la integración de modelos de lenguaje de código abierto desplegados localmente —tales como Llama 3, Mistral o Qwen— como alternativa al modelo externo Gemini 2.5 Flash. Esta evaluación permitiría cuantificar el costo en exactitud frente a la ganancia en soberanía de datos, la eliminación de la dependencia de un proveedor único y la reducción de costos operativos a largo plazo. Touvron et al. (2023) sugieren que los modelos abiertos contemporáneos alcanzan rendimientos competitivos en tareas de clasificación, y la

realizacion de un benchmark comparativo sobre el mismo corpus de 200 casos constituiria una contribucion adicional significativa al estado del arte.

11. Consideraciones éticas y aspectos legales

11.1. Marco normativo aplicable

El sistema desarrollado se enmarca normativamente en la Ley 25.326 de Protección de los Datos Personales de la República Argentina (Honorable Congreso de la Nación Argentina, 2000), en su decreto reglamentario y en las disposiciones complementarias emitidas por la Agencia de Acceso a la Información Pública en su carácter de autoridad de aplicación. Dado que el sistema realiza transferencia de fragmentos textuales hacia infraestructura de inferencia operada por proveedores con sede en los Estados Unidos, resultan aplicables las consideraciones del artículo 12 de la mencionada ley sobre transferencia internacional de datos personales y, de manera complementaria, los lineamientos del Reglamento General de Protección de Datos de la Unión Europea (Parlamento Europeo y Consejo de la Unión Europea, 2016) en aquellos casos que pudieran involucrar a residentes europeos. Argentina mantiene un régimen de adecuación reconocido por la Comisión Europea, lo cual facilita los flujos transfronterizos bidireccionales bajo determinadas condiciones.

11.2. Principios aplicados al tratamiento de datos

El diseño del sistema se atiene de manera explícita a cinco principios fundamentales del tratamiento de datos personales. El principio de licitud establece que todo tratamiento debe ampararse en una base jurídica válida; en este caso, dicha base se encuentra en el legítimo interés de la organización para la gestión interna de los incidentes informáticos reportados por sus propios usuarios en ejercicio de sus funciones laborales. El principio de finalidad determinada limita el uso de los datos a la finalidad declarada de registro y derivación de incidentes, prohibiendo cualquier uso secundario no compatible. El principio de minimización exige recolectar únicamente los datos estrictamente necesarios, razón por la cual el sistema no almacena identificadores personales más allá del nombre de usuario corporativo y descarta cualquier dato adicional capturado incidentalmente. El principio de exactitud se materializa mediante la posibilidad ofrecida al usuario de revisar y corregir el contenido del incidente antes de su persistencia definitiva. El principio de limitación del plazo de conservación se traduce en una política de retención de 90 días para los registros operativos y de un año para los datos de incidentes resueltos.

11.3. Transferencia internacional y pseudonimizacion

La utilizacion del modelo Gemini 2.5 Flash implica la transmision de fragmentos textuales descriptivos de los incidentes hacia infraestructura del proveedor con sede en los Estados Unidos. Para mitigar el riesgo asociado a esta transferencia, el sistema aplica un procedimiento de pseudonimizacion previa a la transmision, mediante el cual se reemplazan automaticamente los nombres propios, las direcciones de correo electronico, los numeros telefonicos, los identificadores corporativos y los nombres de hosts internos por etiquetas genericas tales como [PERSONA], [EMAIL] o [HOST]. Esta pseudonimizacion se realiza dentro del modulo Python desplegado en infraestructura local de la organizacion y se valida mediante un conjunto de expresiones regulares especificas del dominio, acompanadas de pruebas unitarias dedicadas. Adicionalmente, la organizacion ha suscripto el acuerdo de procesamiento de datos provisto por el proveedor del servicio de inferencia.

11.4. Seguridad tecnica

Las medidas de seguridad tecnica implementadas se sustentan en el modelo de la triada confidencialidad, integridad y disponibilidad (CIA) descrito por Stallings (2017). La confidencialidad se garantiza mediante cifrado en transito sobre TLS 1.3 entre todos los componentes del sistema y mediante cifrado en reposo de la base de datos PostgreSQL utilizando la extension pgcrypto para los campos sensibles. La integridad se asegura mediante el uso de identificadores firmados con HMAC-SHA-256, la validacion estricta de esquemas mediante Pydantic en cada llamada a la interfaz REST, y un registro de auditoria de toda modificacion que conserva el actor, la marca temporal y el delta del cambio. La disponibilidad se respalda mediante respaldos diarios automaticos con retencion escalonada, despliegue en contenedores con politicas de reinicio automatico ante fallas, y monitoreo de salud activo mediante endpoints de chequeo. Las credenciales de servicios externos se gestionan exclusivamente mediante variables de entorno inyectadas por el orquestador de contenedores y nunca se almacenan en el repositorio de codigo fuente.

11.5. Consentimiento, derechos del usuario y supervision humana

Los usuarios de la organizacion son informados, mediante una politica de privacidad accesible desde los canales de entrada, sobre las características del sistema, los datos que se recolectan, la transferencia internacional aplicable y los derechos que les asisten conforme a

la normativa vigente. Se garantiza el ejercicio de los derechos de acceso, rectificación, actualización y supresión (derechos ARCO) mediante un procedimiento documentado a cargo del responsable de protección de datos designado por la organización.

En lo que respecta al consentimiento informado para los fines de investigación académica, los 120 empleados de la organización fueron notificados mediante comunicación interna formal, con descripción explícita del propósito del estudio, del tipo de datos tratados, del carácter pseudonimizado de los registros y del derecho a requerir la exclusión de sus incidentes del corpus de validación. El procedimiento fue supervisado por la dirección académica del trabajo y avalado institucionalmente mediante una carta de autorización emitida por la organización adoptante.

En relación con el protocolo de observación naturalista aplicado durante la fase experimental —en el cual los operadores no fueron informados en tiempo real de que sus tiempos de registro estaban siendo medidos, con el objeto de eliminar el efecto Hawthorne—, corresponde dejar constancia de que, una vez concluida la recolección de datos, se realizó una sesión de esclarecimiento posterior (*debriefing*) con los dos operadores participantes. En dicha sesión se explicó el propósito del estudio, se detalló el procedimiento de medición empleado y se ofreció a cada operador la posibilidad de solicitar la exclusión de sus registros individuales del corpus de validación. Ninguno de los operadores ejerció este derecho. Este procedimiento se alinea con las recomendaciones de la Asociación Americana de Psicología (APA, 2017) para investigaciones que emplean observación sin conocimiento previo de los participantes, y fue supervisado por el director del trabajo.

En concordancia con las recomendaciones contemporáneas sobre sistemas que incorporan inteligencia artificial, se preserva en todo momento la posibilidad de supervisión humana significativa. Ninguna decisión que afecte derechos sustanciales de los usuarios se toma de manera completamente automatizada sin posibilidad de revisión, y el sector responsable conserva la potestad de modificar la clasificación inicial cuando lo considere apropiado, registrándose tal modificación en la tabla de trazabilidad descrita en el Capítulo 5. Este mecanismo materializa el principio de la persona en el bucle (*human in the loop*) como salvaguarda frente a errores potenciales del clasificador automático.

Referencias bibliograficas

- Apache Software Foundation. (2024). *Apache Airflow Documentation (Version 2.10)* [Documentación técnica]. <https://airflow.apache.org/docs/>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 1877-1901.
- Cohen, J. (1960). A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement*, 20(1), 37-46. <https://doi.org/10.1177/001316446002000104>
- Crispin, L., & Gregory, J. (2009). *Agile Testing: A Practical Guide for Testers and Agile Teams*. Addison-Wesley.
- Date, C. J. (2003). *An Introduction to Database Systems* (8.^a ed.). Addison-Wesley.
- Fielding, R. T., & Taylor, R. N. (2002). Principled Design of the Modern Web Architecture. *ACM Transactions on Internet Technology*, 2(2), 115-150. <https://doi.org/10.1145/514183.514185>
- Galup, S. D., Dattero, R., Quan, J. J., & Conger, S. (2009). An Overview of IT Service Management. *Communications of the ACM*, 52(5), 124-127. <https://doi.org/10.1145/1506409.1506439>
- Gómez, L., Bustos, C., & Sevilla, G. (2026). Automatización de Mesa de Ayuda N8N (Version 1.0.0) [Software de computadora. GitHub]. <https://github.com/lucaGomezB/Automatizacion-de-Mesa-de-Ayuda-N8N>
- Hernández Sampieri, R., & Mendoza Torres, C. P. (2018). *Metodología de la investigación: Las rutas cuantitativa, cualitativa y mixta*. McGraw-Hill.
- Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.
- Honorable Congreso de la Nación Argentina. (2000, octubre). Ley 25.326 de Protección de los Datos Personales [Boletín Oficial de la República Argentina]. <http://servicios.infoleg.gob.ar/infolegInternet/anexos/60000-64999/64790/norma.htm>

- Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition* (3.^a ed.) [Borrador]. Pearson.
- Ladas, C. (2009). *Scrumban: Essays on Kanban Systems for Lean Software Development*. Modus Cooperandi Press.
- n8n GmbH. (2024). *n8n Documentation* [Documentación técnica oficial]. <https://docs.n8n.io/>
- Office of Government Commerce. (2011). *ITIL Service Operation* (2011.^a ed.). The Stationery Office.
- Paramesh, S. P., & Shreedhara, K. S. (2019). Automated IT Service Desk Systems Using Machine Learning Techniques. En *Lecture Notes in Networks and Systems* (pp. 331-346, Vol. 43). Springer. https://doi.org/10.1007/978-981-13-2514-4_28
- Parlamento Europeo y Consejo de la Unión Europea. (2016, abril). Reglamento (UE) 2016/679 — Reglamento General de Protección de Datos [Diario Oficial de la Unión Europea, L 119/1].
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Powers, D. M. W. (2011). Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation. *Journal of Machine Learning Technologies*, 2(1), 37-63.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9.^a ed.). McGraw-Hill.
- Rabiner, L., & Juang, B.-H. (1993). *Fundamentals of Speech Recognition*. Prentice Hall.
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust Speech Recognition via Large-Scale Weak Supervision. *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, 28492-28518.
- Revina, A., Buza, K., & Meister, V. G. (2020). IT Ticket Classification: The Simpler, the Better. *IEEE Access*, 8, 193380-193395. <https://doi.org/10.1109/ACCESS.2020.3032840>
- Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.^a ed.). Pearson.

- Sokolova, M., & Lapalme, G. (2009). A Systematic Analysis of Performance Measures for Classification Tasks. *Information Processing & Management*, 45(4), 427-437. <https://doi.org/10.1016/j.ipm.2009.03.002>
- Stallings, W. (2017). *Computer Security: Principles and Practice* (4.^a ed.). Pearson.
- The PostgreSQL Global Development Group. (2024). *PostgreSQL 15 Documentation* [Documentacion tecnica oficial]. <https://www.postgresql.org/docs/15/>
- Tiangolo, S. R. (2024). *FastAPI Documentation* [Documentacion tecnica oficial]. <https://fastapi.tiangolo.com/>
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Canton-Ferrer, C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., ... Scialom, T. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models [arXiv:2307.09288]. *arXiv preprint*. <https://doi.org/10.48550/arXiv.2307.09288>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 5998-6008.
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17(3), 261-272. <https://doi.org/10.1038/s41592-019-0686-2>
- Wiggins, A. (2017). The Twelve-Factor App [Manifiesto de arquitectura de software]. <https://12factor.net/>
- Wilson, E. B. (1927). Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158), 209-212. <https://doi.org/10.1080/01621459.1927.10502953>

A. Diagrama de arquitectura del sistema

El Anexo A reúne los diagramas de la arquitectura propuesta en notación UML. Incluye:

- **Diagrama de despliegue:** los cinco componentes principales (N8N, FastAPI, PostgreSQL, Gemini 2.5 Flash, Twilio) y sus relaciones de comunicación cifrada sobre TLS 1.3.
- **Diagrama de secuencia:** flujo extremo a extremo de un incidente desde su recepción en el canal de correo electrónico hasta la confirmación al usuario, incluyendo las interacciones con el clasificador híbrido y la base de datos.
- **Diagrama de componentes:** organización interna del módulo Python por capas (routes, services, repositories, models, classifiers), con sus dependencias y relaciones.

Los diagramas se mantienen en formato editable (`.drawio`) en el repositorio del proyecto para facilitar su actualización futura. La versión en alta resolución se encuentra disponible en el directorio `docs/` del repositorio (Gómez et al. 2026).

B. Repositorio de código fuente

El código fuente del módulo Python, los scripts de migración de la base de datos PostgreSQL gestionados mediante Alembic, los archivos de configuración del despliegue en contenedores Docker y los flujos de integración continua sobre GitHub Actions se encuentran disponibles en el repositorio público del proyecto (Gómez et al. 2026). El commit de referencia para la versión entregada al jurado corresponde al tag `v1.0.0`.

La estructura del repositorio sigue el patrón estándar con directorios separados para el módulo de procesamiento (`Gestion_Incidentes/`), la definición del flujo de orquestación (`Automatizacion_Mesa_de_Ayuda.json`), las migraciones de base de datos (`alembic/`), las pruebas automatizadas (`tests/`), los artefactos de evaluación (`evaluation/`) y la documentación operativa (`docs/`). El archivo `README.md` incluye instrucciones detalladas para el despliegue local en menos de 15 minutos.

C. Esquema de base de datos

El esquema completo de la base de datos PostgreSQL se documenta mediante scripts SQL versionados que definen las cinco tablas descritas en la Tabla 4 del Capítulo 5, las restricciones de integridad referencial (claves foráneas con eliminación en cascada condicional),

los índices secundarios sobre los atributos de búsqueda frecuente (fecha de creación, sector responsable y estado actual del ticket), y las funciones almacenadas que generan los identificadores secuenciales con prefijo configurable. Las migraciones incrementales se gestionan mediante Alembic, lo que permite reproducir el estado de la base en cualquier punto histórico del desarrollo y revertir cambios cuando resulte necesario.

La migración inicial incluye la creación de las tablas de catálogo (`sector`, `estado`, `canal_origen`) con sus valores semilla, garantizando que un despliegue nuevo del sistema comience con los datos de referencia correctos. Las migraciones subsiguientes documentan la evolución del esquema durante las seis iteraciones de desarrollo descritas en el Capítulo 4.

D. Especificación OpenAPI de la interfaz REST

La especificación completa de la interfaz REST se genera automáticamente mediante FastAPI conforme a la versión 3.1 de la especificación OpenAPI (Tiangolo, 2024). El documento resultante describe los puntos de entrada listados en la Tabla 5 del Capítulo 5, los esquemas de los cuerpos de solicitud y respuesta validados mediante Pydantic v2, los códigos de estado posibles para cada método HTTP, y ejemplos representativos para cada caso de uso.

La interfaz se encuentra disponible adicionalmente en formato interactivo a través del componente Swagger UI integrado en el servidor, accesible bajo la ruta `/docs` durante la ejecución, y en formato estático como archivo `openapi.json` en el directorio `docs/` del repositorio. La sincronización entre el código y la especificación estática se verifica automáticamente en el pipeline de integración continua.

E. Configuración del flujo N8N

La exportación del flujo de orquestación en formato JSON incluye la totalidad de los 12 nodos configurados, sus parámetros operativos, las credenciales referenciadas mediante identificadores opacos sin exposición de valores, y los conectores entre nodos. El archivo `Automatizacion_Mesa_de_Ayuda.json` en la raíz del repositorio contiene esta exportación. La importación de este artefacto en una instancia limpia de N8N 1.62 reproduce íntegramente el comportamiento operativo del sistema, sujeta únicamente a la provisión de las credenciales correspondientes a los servicios externos integrados (servidor IMAP de correo, cuenta Twilio y clave de API de Gemini) mediante el sistema de credenciales nativo de la plataforma.

F. Corpus de validacion

El corpus de 200 incidentes utilizado para la validacion experimental se conserva en formato CSV en el directorio `evaluation/data/` del repositorio, con columnas para el identificador anonimo del caso, la descripcion cruda pseudonimizada, el canal de origen, la categoria asignada por consenso humano, la categoria predicha por el clasificador, el tiempo de registro medido en cada flujo y la confianza asociada a la prediccion. Se incluyen ademas las planillas de cronometraje del flujo manual y los registros automaticos generados por el flujo automatizado.

Por razones de privacidad y en cumplimiento de la Ley 25.326 (Honorable Congreso de la Nación Argentina, 2000), las descripciones textuales han sido pseudonimizadas previamente a su inclusion en el repositorio publico, reemplazando nombres propios, direcciones de correo y otros identificadores personales por etiquetas genericas.

G. Documentacion operativa

La documentacion operativa del sistema describe los procedimientos de despliegue inicial, configuracion recurrente, respaldo y restauracion, monitoreo activo y respuesta ante incidentes operativos. Se incluyen instrucciones detalladas para los entornos de desarrollo, preproduccion y produccion, asi como una guia de resolucion de incidencias frecuentes orientada al personal de operaciones de la organizacion adoptante. La documentacion se mantiene actualizada en el repositorio del proyecto y se versiona conjuntamente con el codigo fuente, garantizando coherencia entre la version desplegada y los procedimientos vigentes en cada momento del ciclo de vida del sistema.